

- 1. Chapter 9: Objects and Classes (对象和类)
 - 1.1. Motivations (动机)
 - 1.2. Objectives (目标)
 - 1.3. OO Programming Concepts (面向对象编程概念)
 - 1.4. Objects (对象)
 - 1.5. Classes (类)
 - 1.6. Classes (类) - Continued
 - 1.7. UML Class Diagram (UML 类图)
 - 1.8. Constructors (构造函数)
 - 1.9. Creating Objects Using Constructors (使用构造函数创建对象)
 - 1.10. Default Constructor (默认构造函数)
 - 1.11. Declaring Object Reference Variables (声明对象引用变量)
 - 1.12. Declaring/Creating Objects in a Single Step (一步声明/创建对象)
 - 1.13. Accessing Object's Members (访问对象的成员)
 - 1.14. Reference Data Fields (引用数据字段)
 - 1.15. Default Value for a Data Field (数据字段的默认值)
 - 1.16. Differences between Variables of Primitive Data Types and Object Types (基本数据类型变量和对象类型变量的区别)
 - 1.17. Copying Variables of Primitive Data Types and Object Types (复制基本数据类型变量和对象类型变量)
 - 1.18. Garbage Collection (垃圾回收)
 - 1.19. Java Library Classes (Java 库类)
 - 1.20. Instance vs. Static (实例 vs. 静态)
 - 1.21. Visibility Modifiers (可见性修饰符)
 - 1.22. Why Data Fields Should Be private? (为什么数据字段应该是私有的?)
 - 1.23. Passing Objects to Methods (向方法传递对象)
 - 1.24. Array of Objects (对象数组)
 - 1.25. Immutable Objects and Classes (不可变对象和类)
 - 1.26. Scope of **Variables** (变量的作用域)
 - 1.27. The `this` Keyword (this 关键字)
- 2. Chapter 10: Thinking in Objects (面向对象思考)
 - 2.1. Motivations (动机)
 - 2.2. Objectives (目标)
 - 2.3. Class Abstraction and Encapsulation (类抽象和封装)

- 2.4. Designing the Loan Class (设计贷款类)
- 2.5. Object-Oriented Thinking (面向对象思维)
- 2.6. The BMI Class (BMI 类)
- 2.7. Class Relationships (类关系)
- 2.8. Class Representation (类表示)
- 2.9. Aggregation or Composition (聚合或组合)
- 2.10. Aggregation Between Same Class (同类间的聚合)
- 2.11. Example: The Course Class (示例：课程类)
- 2.12. Example: The StackOfIntegers Class (示例：整数栈类)
- 2.13. Implementing StackOfIntegers Class (实现整数栈类)
- 2.14. Wrapper Classes (包装类)
- 2.15. The Integer and Double Classes (Integer 和 Double 类)
- 2.16. Automatic Conversion Between Primitive Types and Wrapper Class Types (基本类型与包装类类型之间的自动转换)
- 2.17. BigInteger and BigDecimal (大整数和大数据)
- 2.18. The String Class (String 类)
- 2.19. Strings Are Immutable (字符串是不可变的)
- 2.20. Interned Strings (内部化字符串)
- 2.21. Replacing and Splitting Strings (替换和分割字符串)
- 2.22. Matching, Replacing and Splitting by Patterns (按模式匹配、替换和分割)
- 2.23. Convert Character and Numbers to Strings (字符和数字转换为字符串)
- 2.24. StringBuilder and StringBuffer (StringBuilder 和 StringBuffer)
- 2.25. Modifying Strings in the Builder (在 StringBuilder 中修改字符串)
- 2.26. The `toString`, `capacity`, `length`, `setLength`, and `charAt` Methods (`toString`、`capacity`、`length`、`setLength` 和 `charAt` 方法)
- 3. Chapter 11: Inheritance and Polymorphism (继承与多态)
 - 3.1. Superclasses and Subclasses (超类和子类)
 - 3.2. Are superclass's Constructor Inherited? (超类的构造函数是否被继承?)
 - 3.3. Constructor Chaining (构造函数链)
 - 3.4. Defining a Subclass (定义子类)
 - 3.5. Calling Superclass Methods (调用超类方法)
 - 3.6. Overriding Methods in the Superclass (重写超类中的方法)
 - 3.7. Overriding vs. Overloading (重写 vs. 重载)
 - 3.8. The `Object` Class and Its Methods (`Object` 类及其方法)
 - 3.9. Polymorphism (多态)
 - 3.10. Dynamic Binding (动态绑定)
 - 3.11. Method Matching vs. Binding (方法匹配 vs. 绑定)
 - 3.12. Generic Programming (泛型编程)

- 3.13. Casting Objects (对象类型转换)
- 3.14. The `instanceof` Operator (`instanceof` 运算符)
- 3.15. The `ArrayList` Class (`ArrayList` 类)
- 3.16. Generic Type (泛型类型)
- 3.17. Differences and Similarities between Arrays and `ArrayList` (数组和 `ArrayList` 的区别与相似之处)
- 3.18. The `MyStack` Classes (`MyStack` 类)
- 3.19. The `protected` Modifier (`protected` 修饰符)
- 3.20. Accessibility Summary (可访问性摘要)
- 3.21. The `final` Modifier (`final` 修饰符)
- 4. Chapter 12: Exception Handling and Text IO (异常处理和文本I/O)
 - 4.1. Motivations (动机)
 - 4.2. Objectives (目标)
 - 4.3. 异常处理
 - 4.4. Exception-Handling Overview (异常处理概述)
 - 4.5. Exception Advantages (异常优势)
 - 4.6. Handling `InputMismatchException` (处理 `InputMismatchException`)
 - 4.7. Exception Types (异常类型)
 - 4.8. System Errors (系统错误)
 - 4.9. Exceptions (异常)
 - 4.10. Runtime Exceptions (运行时异常)
 - 4.11. Checked Exceptions vs. Unchecked Exceptions (受检异常 vs. 非受检异常)
 - 4.12. Unchecked Exceptions (非受检异常)
 - 4.13. Declaring, Throwing, and Catching Exceptions (声明、抛出和捕获异常)
 - 4.14. Declaring Exceptions (声明异常)
 - 4.15. Throwing Exceptions (抛出异常)
 - 4.16. Throwing Exceptions Example (抛出异常示例)
 - 4.17. Catching Exceptions (捕获异常)
 - 4.18. Catch or Declare Checked Exceptions (捕获或声明受检异常)
 - 4.19. Example: Declaring, Throwing, and Catching Exceptions (示例：声明、抛出和捕获异常)
 - 4.20. Rethrowing Exceptions (重新抛出异常)
 - 4.21. The `finally` Clause (`finally` 子句)
 - 4.22. Trace a Program Execution (追踪程序执行)
 - 4.23. Cautions When Using Exceptions (使用异常时的注意事项)
 - 4.24. When to Throw Exceptions (何时抛出异常)
 - 4.25. When to Use Exceptions (何时使用异常)
 - 4.26. Defining Custom Exception Classes (定义自定义异常类)

- 4.27. Custom Exception Class Example (自定义异常类示例)
- 4.28. The File Class (File 类)
- 4.29. Obtaining file properties and manipulating file (获取文件属性和操作文件)
- 4.30. Problem: Explore File Properties (问题：探索文件属性)
- 4.31. Text I/O (文本I/O)
- 4.32. Writing Data Using PrintWriter (使用 PrintWriter 写入数据)
- 4.33. Try-with-resources (try-with-resources)
- 4.34. Reading Data Using Scanner (使用 Scanner 读取数据)
- 4.35. Problem: Replacing Text (问题：替换文本)
- 4.36. Reading Data from the Web (从 Web 读取数据)
- 4.37. Case Study: Web Crawler (案例研究：网络爬虫)
- 5. Chapter 13: Abstract Classes and Interfaces (抽象类和接口)
 - 5.1. Motivations (动机)
 - 5.2. Objectives (目标)
 - 5.3. Abstract Classes and Abstract Methods (抽象类和抽象方法)
 - 5.4. Abstract method in abstract class (抽象类中的抽象方法)
 - 5.5. Object cannot be created from abstract class (抽象类不能创建对象)
 - 5.6. Abstract class without abstract method (没有抽象方法的抽象类)
 - 5.7. Superclass of abstract class may be concrete (抽象类的超类可以是具体类)
 - 5.8. Concrete method overridden to be abstract (具体方法被重写为抽象方法)
 - 5.9. Abstract class as type (抽象类作为类型)
 - 5.10. Case Study: the Abstract Number Class (案例研究：抽象 Number 类)
 - 5.11. The Abstract Calendar Class and Its GregorianCalendar subclass (抽象 Calendar 类及其 GregorianCalendar 子类)
 - 5.12. The `get` Method in Calendar Class (Calendar 类中的 `get` 方法)
 - 5.13. Interfaces (接口)
 - 5.14. What is an interface? Why is an interface useful? (什么是接口？为什么接口有用？)
 - 5.15. Define an Interface (定义接口)
 - 5.16. Interface is a Special Class (接口是一种特殊类)
 - 5.17. Omitting Modifiers in Interfaces (接口中省略修饰符)
 - 5.18. Example: The Comparable Interface (示例：Comparable 接口)
 - 5.19. The `toString`, `equals`, and `hashCode` Methods (`toString`、`equals` 和 `hashCode` 方法)
 - 5.20. Integer and BigInteger Classes (Integer 和 BigInteger 类)
 - 5.21. The `Cloneable` Interfaces (Cloneable 接口)
 - 5.22. Shallow vs. Deep Copy (浅拷贝 vs. 深拷贝)
 - 5.23. Interfaces vs. Abstract Classes (接口 vs. 抽象类)
 - 5.24. Interfaces vs. Abstract Classes, cont. (接口 vs. 抽象类，续)
 - 5.25. Caution: conflict interfaces (注意：冲突接口)

- 5.26. Whether to use an interface or a class? (何时使用接口或类?)
- 5.27. The Rational Class (有理数类)
- 5.28. Designing a Class (设计类)
- 5.29. Using Visibility Modifiers (使用可见性修饰符)
- 5.30. Using the `static` Modifier (使用 `static` 修饰符)
- 6. Chapter 14: JavaFX Basics (JavaFX 基础)
 - 6.1. Motivations (动机)
 - 6.2. Objectives (目标)
 - 6.3. JavaFX vs Swing and AWT (JavaFX 与 Swing 和 AWT 对比)
 - 6.4. 创建 JavaFX 项目 (Creating JavaFX Projects)
 - 6.5. Basic Structure of JavaFX (JavaFX 基本结构)
 - 6.6. Panes, UI Controls, and Shapes (窗格、UI 控件和形状)
 - 6.7. Display a Shape (显示形状)
 - 6.8. Binding Properties (绑定属性)
 - 6.9. Uni/Bidirectional Binding (单向/双向绑定)
 - 6.10. Common Properties and Methods for Nodes (节点的通用属性和方法)
 - 6.11. The Color Class (Color 类)
 - 6.12. The Font Class (Font 类)
 - 6.13. The Image Class (Image 类)
 - 6.14. The ImageView Class (ImageView 类)
 - 6.15. Layout Panes (布局窗格)
 - 6.16. FlowPane (流式窗格)
 - 6.17. GridPane (网格窗格)
 - 6.18. BorderPane (边框窗格)
 - 6.19. HBox (水平箱式窗格)
 - 6.20. VBox (垂直箱式窗格)
 - 6.21. Shapes (形状)
 - 6.22. Text (文本)
 - 6.23. Line (线条)
 - 6.24. Rectangle (矩形)
 - 6.25. Circle (圆形)
 - 6.26. Ellipse (椭圆)
 - 6.27. Arc (弧)
 - 6.28. Polygon and Polyline (多边形和多段线)
 - 6.29. Case Study: The ClockPane Class (案例研究: 时钟窗格类)
- 7. Chapter 15: Event-Driven Programming and Animations (事件驱动编程和动画)
 - 7.1. Motivations (动机)
 - 7.2. Objectives (目标)

- 7.3. Procedural vs. Event-Driven Programming (过程式 vs. 事件驱动编程)
- 7.4. Taste of Event-Driven Programming (事件驱动编程初体验)
- 7.5. Handling GUI Events (处理 GUI 事件)
- 7.6. Trace Execution (追踪执行)
- 7.7. Events (事件)
- 7.8. Event Classes (事件类)
- 7.9. Event Information (事件信息)
- 7.10. Selected User Actions and Handlers (选定的用户操作和处理程序)
- 7.11. The Delegation Model (委托模型)
- 7.12. The Delegation Model: Example (委托模型：示例)
- 7.13. Example: First Version for ControlCircle (no listeners) (示例：ControlCircle 的第一个版本（无监听器）)
- 7.14. Example: Second Version for ControlCircle (with listener for Enlarge) (示例：ControlCircle 的第二个版本（带放大监听器）)
- 7.15. Inner Class Listeners (内部类监听器)
- 7.16. Inner Classes (内部类)
- 7.17. Inner Classes, cont. (内部类，续)
- 7.18. Anonymous Inner Classes (匿名内部类)
- 7.19. Simplifying Event Handling Using Lambda Expressions (使用 Lambda 表达式简化事件处理)
- 7.20. Basic Syntax for a Lambda Expression (Lambda 表达式基本语法)
- 7.21. The `MouseEvent` Class (`MouseEvent` 类)
- 7.22. The `KeyEvent` Class (`KeyEvent` 类)
- 7.23. The `KeyCode` Constants (`KeyCode` 常量)
- 7.24. Listeners for Observable Objects (可观察对象的监听器)
- 7.25. Animation (动画)
- 7.26. `PathTransition` (路径过渡)
- 7.27. `FadeTransition` (淡入淡出过渡)
- 7.28. `Timeline` (时间轴)
- 7.29. Case Study: Bouncing Ball (案例研究：弹跳球)
- 8. Chapter 16: JavaFX UI Controls and Multimedia (JavaFX UI 控件和多媒体)
 - 8.1. Motivations (动机)
 - 8.2. Objectives (目标)
 - 8.3. Frequently Used UI Controls (常用 UI 控件)
 - 8.4. Labeled (带标签的控件)
 - 8.5. `Label` (标签)
 - 8.6. `ButtonBase` and `Button` (按钮基类和按钮)
 - 8.7. `CheckBox` (复选框)

- 8.8. RadioButton (单选按钮)
- 8.9. TextField (文本字段)
- 8.10. TextArea (文本区域)
- 8.11. ComboBox (组合框)
- 8.12. ListView (列表视图)
- 8.13. ScrollBar (滚动条)
- 8.14. Slider (滑块)
- 8.15. Case Study: TicTacToe (案例研究: 井字棋)
- 8.16. Media (媒体)
- 9. Chapter 14: JavaFX Basics (JavaFX 基础)
 - 9.1. Motivations (动机)
 - 9.2. Objectives (目标)
 - 9.3. JavaFX vs Swing and AWT (JavaFX 与 Swing 和 AWT 对比)
 - 9.4. 创建 JavaFX 项目 (Creating JavaFX Projects)
 - 9.5. Basic Structure of JavaFX (JavaFX 基本结构)
 - 9.6. Panes, UI Controls, and Shapes (窗格、UI 控件和形状)
 - 9.7. Display a Shape (显示形状)
 - 9.8. Binding Properties (绑定属性)
 - 9.9. Uni/Bidirectional Binding (单向/双向绑定)
 - 9.10. Common Properties and Methods for Nodes (节点的通用属性和方法)
 - 9.11. The Color Class (Color 类)
 - 9.12. The Font Class (Font 类)
 - 9.13. The Image Class (Image 类)
 - 9.14. The ImageView Class (ImageView 类)
 - 9.15. Layout Panes (布局窗格)
 - 9.16. FlowPane (流式窗格)
 - 9.17. GridPane (网格窗格)
 - 9.18. BorderPane (边框窗格)
 - 9.19. HBox (水平箱式窗格)
 - 9.20. VBox (垂直箱式窗格)
 - 9.21. Shapes (形状)
 - 9.22. Text (文本)
 - 9.23. Line (线条)
 - 9.24. Rectangle (矩形)
 - 9.25. Circle (圆形)
 - 9.26. Ellipse (椭圆)
 - 9.27. Arc (弧)
 - 9.28. Polygon and Polyline (多边形和多段线)

- 9.29. Case Study: The ClockPane Class (案例研究：时钟窗格类)
- 10. Chapter 15: Event-Driven Programming and Animations (事件驱动编程和动画)
 - 10.1. Motivations (动机)
 - 10.2. Objectives (目标)
 - 10.3. Procedural vs. Event-Driven Programming (过程式 vs. 事件驱动编程)
 - 10.4. Taste of Event-Driven Programming (事件驱动编程初体验)
 - 10.5. Handling GUI Events (处理 GUI 事件)
 - 10.6. Trace Execution (追踪执行)
 - 10.7. Events (事件)
 - 10.8. Event Classes (事件类)
 - 10.9. Event Information (事件信息)
 - 10.10. Selected User Actions and Handlers (选定的用户操作和处理程序)
 - 10.11. The Delegation Model (委托模型)
 - 10.12. The Delegation Model: Example (委托模型：示例)
 - 10.13. Example: First Version for ControlCircle (no listeners) (示例：ControlCircle 的第一个版本（无监听器）)
 - 10.14. Example: Second Version for ControlCircle (with listener for Enlarge) (示例：ControlCircle 的第二个版本（带放大监听器）)
 - 10.15. Inner Class Listeners (内部类监听器)
 - 10.16. Inner Classes (内部类)
 - 10.17. Inner Classes, cont. (内部类，续)
 - 10.18. Anonymous Inner Classes (匿名内部类)
 - 10.19. Simplifying Event Handling Using Lambda Expressions (使用 Lambda 表达式简化事件处理)
 - 10.20. Basic Syntax for a Lambda Expression (Lambda 表达式基本语法)
 - 10.21. The `MouseEvent` Class (MouseEvent 类)
 - 10.22. The `KeyEvent` Class (KeyEvent 类)
 - 10.23. The `KeyCode` Constants (KeyCode 常量)
 - 10.24. Listeners for Observable Objects (可观察对象的监听器)
 - 10.25. Animation (动画)
 - 10.26. PathTransition (路径过渡)
 - 10.27. FadeTransition (淡入淡出过渡)
 - 10.28. Timeline (时间轴)
 - 10.29. Case Study: Bouncing Ball (案例研究：弹跳球)
- 11. Chapter 16: JavaFX UI Controls and Multimedia (JavaFX UI 控件和多媒体)
 - 11.1. Motivations (动机)
 - 11.2. Objectives (目标)
 - 11.3. Frequently Used UI Controls (常用 UI 控件)

- 11.4. Labeled (带标签的控件)
- 11.5. Label (标签)
- 11.6. ButtonBase and Button (按钮基类和按钮)
- 11.7. CheckBox (复选框)
- 11.8. RadioButton (单选按钮)
- 11.9. TextField (文本字段)
- 11.10. TextArea (文本区域)
- 11.11. ComboBox (组合框)
- 11.12. ListView (列表视图)
- 11.13. ScrollBar (滚动条)
- 11.14. Slider (滑块)
- 11.15. Case Study: TicTacToe (案例研究：井字棋)
- 11.16. Media (媒体)
- 12. Chapter 17: Binary I/O (二进制 I/O)
 - 12.1. Motivations (动机)
 - 12.2. Objectives (目标)
 - 12.3. How is I/O Handled in Java? (Java 中如何处理 I/O?)
 - 12.4. Text File vs. Binary File (文本文件 vs. 二进制文件)
 - 12.5. Binary I/O (二进制 I/O)
 - 12.6. Binary I/O Classes (二进制 I/O 类)
 - 12.7. InputStream (输入流)
 - 12.8. OutputStream (输出流)
 - 12.9. FileInputStream/FileOutputStream (文件输入/输出流)
 - 12.10. FilterInputStream/FilterOutputStream (过滤输入/输出流)
 - 12.11. DataInputStream/DataOutputStream (数据输入/输出流)
 - 12.12. DataInputStream (数据输入流)
 - 12.13. DataOutputStream (数据输出流)
 - 12.14. Characters and Strings in Binary I/O (二进制 I/O 中的字符和字符串)
 - 12.15. Using DataInputStream/DataOutputStream (使用数据输入/输出流)
 - 12.16. Concept of pipe line (管道概念)
 - 12.17. Order and Format (顺序和格式)
 - 12.18. Checking End of File (检查文件末尾)
 - 12.19. BufferedInputStream/BufferedOutputStream (缓冲输入/输出流)
 - 12.20. Constructing BufferedInputStream/BufferedOutputStream (构造缓冲输入/输出流)
 - 12.21. Case Studies: Copy File (案例研究：复制文件)
 - 12.22. Object I/O (对象 I/O)
 - 12.23. ObjectInputStream (对象输入流)
 - 12.24. ObjectOutputStream (对象输出流)

- 12.25. Using Object Streams (使用对象流)
- 12.26. The Serializable Interface (Serializable 接口)
- 12.27. The `transient` Keyword (transient 关键字)
- 12.28. The `transient` Keyword, cont. (transient 关键字, 续)
- 12.29. Serializing Arrays (序列化数组)
- 12.30. Random Access Files (随机访问文件)
- 12.31. RandomAccessFile (随机访问文件)
- 12.32. File Pointer (文件指针)
- 12.33. RandomAccessFile Methods (随机访问文件方法)
- 12.34. RandomAccessFile Methods, cont. (随机访问文件方法, 续)
- 12.35. RandomAccessFile Constructor (RandomAccessFile 构造函数)
- 13. Chapter 19: Generics (泛型)
 - 13.1. Objectives (目标)
 - 13.2. Why Do You Get a Warning? (为什么会有警告?)
 - 13.3. Fix the Warning (修复警告)
 - 13.4. What is Generics? (什么是泛型?)
 - 13.5. Why Generics? (为什么使用泛型?)
 - 13.6. Generic Type (泛型类型)
 - 13.7. Generic Instantiation (泛型实例化)
 - 13.8. Generic ArrayList in JDK 1.5 (JDK 1.5 中的泛型 ArrayList)
 - 13.9. No Casting Needed (无需类型转换)
 - 13.10. Declaring Generic Classes and Interfaces (声明泛型类和接口)
 - 13.11. Generic Methods (泛型方法)
 - 13.12. Bounded Generic Type (有界泛型类型)
 - 13.13. Raw Type and Backward Compatibility (原始类型和向后兼容性)
 - 13.14. Raw Type is Unsafe (原始类型不安全)
 - 13.15. Make it Safe (使其安全)
 - 13.16. Wildcards (通配符)
 - 13.17. Generic Types and Wildcard Types (泛型类型和通配符类型)
 - 13.18. Avoiding Unsafe Raw Types (避免不安全的原始类型)
 - 13.19. Erasure and Restrictions on Generics (类型擦除和泛型限制)
 - 13.20. Compile Time Checking (编译时检查)
 - 13.21. Important Facts (重要事实)
 - 13.22. Restrictions on Generics (泛型限制)
 - 13.23. Designing Generic Matrix Classes (设计泛型矩阵类)
 - 13.24. UML Diagram (UML 图)
 - 13.25. Source Code (源代码)
- 14. Chapter 20: Lists, Stacks, Queues, and Priority Queues (列表、栈、队列和优先队列)

- 14.1. Objectives (目标)
- 14.2. What is Data Structure? (什么是数据结构?)
- 14.3. Java Collection Framework hierarchy (Java 集合框架层次结构)
- 14.4. Java Collection Framework hierarchy, cont. (Java 集合框架层次结构, 续)
- 14.5. The Collection Interface (Collection 接口)
- 14.6. The List Interface (List 接口)
- 14.7. The List Interface, cont. (List 接口, 续)
- 14.8. The List Iterator (列表迭代器)
- 14.9. ArrayList and LinkedList (ArrayList 和 LinkedList)
- 14.10. The Comparator Interface (Comparator 接口)
- 14.11. Other Comparator Examples (其他比较器示例)
- 14.12. The Collections Class (Collections 类)
- 14.13. The Collections Class UML Diagram (Collections 类 UML 图)
- 14.14. Case Study: Multiple Bouncing Balls (案例研究: 多重弹跳球)
- 14.15. The Vector and Stack Classes (Vector 和 Stack 类)
- 14.16. The Vector Class (Vector 类)
- 14.17. The Vector Class, cont. (Vector 类, 续)
- 14.18. The Stack Class (Stack 类)
- 14.19. Queues and Priority Queues (队列和优先队列)
- 14.20. The Queue Interface (Queue 接口)
- 14.21. Using LinkedList for Queue (使用 LinkedList 作为队列)
- 14.22. The PriorityQueue Class (PriorityQueue 类)
- 14.23. Case Study: Evaluating Expressions (案例研究: 表达式求值)
- 14.24. Algorithm (算法)
- 14.25. Example (示例)

1. Chapter 9: Objects and Classes (对象和类)

1.1. Motivations (动机)

传统的编程方法（如选择、循环、方法和数组）不足以开发复杂的图形用户界面（GUI）和大型软件系统。面向对象编程（OOP）提供了更好的解决方案。

1.2. Objectives (目标)

本章目标包括：描述对象和类，使用 UML 图形表示法，定义类和创建对象，使用构造函数，通过对象引用变量访问对象成员，区分对象引用与基本类型变量，使用 Java 库类（Date、Random、Point2D），区分实例与静态变量/方法，定义私有数据字段及 get / set 方法，封装数据字段，开发带对象参数的方法，在数组中存储和处理对象，创建不可变对象，确定变量作用域，以及使用 this 关键字。

1.3. OO Programming Concepts (面向对象编程概念)

面向对象编程（OOP）涉及使用对象进行编程。

一个对象代表现实世界中可识别的实体，具有唯一的身份、状态和行为。

对象的状态由数据字段（属性）及其当前值组成。

对象的行为由方法定义。

1.4. Objects (对象)

一个对象既有状态也有行为。状态定义了对象，行为定义了对象做什么。

- **类** 是创建对象的模板。
- **对象** 是类的实例，拥有类定义的数据字段（属性）和方法（行为）。

1.5. Classes (类)

类是定义相同类型对象的构造。

Java 类使用变量定义数据字段，使用方法定义行为。

类还提供特殊类型的方法，称为**构造函数**，用于构造对象。

1.6. Classes (类) - Continued

Circle 类的 Java 示例代码结构：

- **数据字段**: double radius = 1.0;
- **构造函数**: Circle() (无参) 和 Circle(double newRadius) (带参)。
- **方法**: double getArea()。

1.7. UML Class Diagram (UML 类图)

UML 类图用于图形化表示类和对象。

一个类在 UML 中通常分为三部分：类名、数据字段和方法。

对象实例的 UML 表示通常只显示对象名和其数据字段的当前值。

1.8. Constructors (构造函数)

构造函数是特殊的方法，用于构造对象。

- **无参构造函数**：没有参数的构造函数。
- 构造函数必须与类同名。
- 构造函数没有返回类型（包括 `void`）。
- 使用 `new` 运算符调用构造函数来初始化对象。

1.9. Creating Objects Using Constructors (使用构造函数创建对象)

语法： `new ClassName();` 或 `new ClassName(arguments);`

1.10. Default Constructor (默认构造函数)

如果类中未显式定义任何构造函数，Java 会自动提供一个带有空体的无参默认构造函数。

1.11. Declaring Object Reference Variables (声明对象引用变量)

要引用对象，需声明一个引用变量。

语法： `ClassName objectRefVar;`

1.12. Declaring/Creating Objects in a Single Step (一步声明/创建对象)

可以将对象声明和创建合并： `ClassName objectRefVar = new ClassName();`

1.13. Accessing Object's Members (访问对象的成员)

使用点运算符（`.`）访问对象的成员。

- **引用数据**： `objectRefVar.data`
- **调用方法**： `objectRefVar.methodName(arguments)`

1.14. Reference Data Fields (引用数据字段)

数据字段可以是引用类型（如 `String`）。

引用类型的数据字段如果没有引用任何对象，会持有特殊值 `null`。

1.15. Default Value for a Data Field (数据字段的默认值)

- 引用类型数据字段默认值是 `null`。
- 数值类型默认值是 `0`。
- 布尔类型默认值是 `false`。
- 字符类型默认值是 `'\u0000'`。
- **注意:** Java 不会为方法内部的局部变量分配默认值，它们必须在使用前显式初始化。

1.16. Differences between Variables of Primitive Data Types and Object Types (基本数据类型变量和对象类型变量的区别)

- **基本类型:** 变量直接存储值。
- **对象类型:** 变量存储的是对象的引用（内存地址）。

1.17. Copying Variables of Primitive Data Types and Object Types (复制基本数据类型变量和对象类型变量)

- **基本类型赋值:** 复制值本身，两个变量独立。
- **对象类型赋值:** 复制引用，两个变量指向同一个对象。

1.18. Garbage Collection (垃圾回收)

当对象不再被任何引用变量指向时，它成为“垃圾”。

Java 虚拟机 (JVM) 会自动回收这些垃圾对象所占用的内存空间。

可以显式地将 `null` 赋给引用变量以帮助 JVM 识别不再需要的对象。

1.19. Java Library Classes (Java 库类)

- `java.util.Date` : 用于处理日期和时间。
- `java.util.Random` : 用于生成伪随机数序列。
- `javafx.geometry.Point2D` : 用于表示二维平面上的点。

1.20. Instance vs. Static (实例 vs. 静态)

- **实例变量和方法:** 属于特定对象（实例），通过对象调用。
- **静态变量和方法:** 由类的所有实例共享，不绑定到特定对象，通过类名调用。
- **静态常量:** 使用 `final static` 声明，由所有实例共享。

1.21. Visibility Modifiers (可见性修饰符)

- **public** : 对任何包中的任何类都可见。
- **private** : 只能被声明它们的类访问。
- **默认 (无修饰符)**: 只能被同一包中的类访问。
- **protected** : 可被同一包中的任何类访问, 或其子类 (即使在不同包中)。

1.22. Why Data Fields Should Be private? (为什么数据字段应该是私有的?)

- 保护数据。
- 使代码易于维护 (封装)。

1.23. Passing Objects to Methods (向方法传递对象)

- **基本类型值传递**: 传递值的副本。
- **引用类型值传递**: 传递对象引用的副本, 方法可以修改原始对象。

1.24. Array of Objects (对象数组)

对象数组实际上是引用变量的数组。

数组中的每个元素都是对实际对象的引用。

1.25. Immutable Objects and Classes (不可变对象和类)

- **不可变对象**: 内容一旦创建就不能改变。
- **不可变类**: 其对象是不可变的类。通常通过将所有数据字段设置为 `private` 且不提供 `set` 方法 (修改器方法) 来实现。
- **注意**: 一个类即使所有数据字段都是 `private` 且没有修改器, 也不一定不可变, 如果它返回了对其内部可变对象的引用。

1.26. Scope of Variables (变量的作用域)

- **实例变量和静态变量**: 作用域是整个类。可在类内部的任何位置声明。
- **局部变量**: 作用域从声明处开始, 持续到包含它的代码块结束。必须在使用前显式初始化。

1.27. The `this` Keyword (this 关键字)

- `this` 关键字是引用对象本身的引用名称。
- 常见用法: 引用类中被隐藏的数据字段 (当局部变量与数据字段同名时)。

- 另一常见用法：允许构造函数调用同类的另一个构造函数（构造函数链）。

2. Chapter 10: Thinking in Objects (面向对象思考)

2.1. Motivations (动机)

本章旨在演示如何使用面向对象范式解决问题，以利用其灵活性和模块化，构建可重用软件。

2.2. Objectives (目标)

本章目标包括：应用类抽象，探索过程式与面向对象范式差异，发现类间关系，设计面向对象程序，使用包装类处理基本值，简化基本类型与包装类自动转换，使用 `BigInteger` 和 `BigDecimal` 处理大数，使用 `String` 处理不可变字符串，以及使用 `StringBuilder` 和 `StringBuffer` 处理可变字符串。

2.3. Class Abstraction and Encapsulation (类抽象和封装)

类抽象：将类实现与类使用分离。

类创建者提供类描述（即契约），用户无需了解具体实现细节。

封装：实现细节对用户隐藏。

类契约：由公共方法签名和公共常量组成，客户端通过契约使用类。

2.4. Designing the Loan Class (设计贷款类)

UML 图展示了 `Loan` 类的数据字段和方法，包括利率、年限、贷款金额、贷款日期等，以及计算每月还款和总还款的方法。

2.5. Object-Oriented Thinking (面向对象思维)

面向对象编程提供比过程式编程更灵活、更模块化的方式，有利于构建可重用软件。

2.6. The BMI Class (BMI 类)

BMI (Body Mass Index) 类示例。

2.7. Class Relationships (类关系)

类之间的主要关系包括：

- **关联 (Association)**: 两个类之间的通用二元关系 (例如, 学生“选修”课程, 教师“教授”课程)。
- **聚合 (Aggregation)**: 一种“has-a”关系, 表示一种弱所有权 (例如, 学生“拥有”姓名, 但姓名可以独立存在)。
- **组合 (Composition)**: 聚合的特殊情况, 表示一种强所有权, 部分不能独立于整体存在 (例如, 学生“拥有”地址, 但地址可能只存在于学生对象中)。
- **继承 (Inheritance)**: 一种“is-a”关系 (在 Chapter 13 中讨论)。

2.8. Class Representation (类表示)

聚合关系通常表示为聚合类中的数据字段。

2.9. Aggregation or Composition (聚合或组合)

许多教材不区分聚合和组合, 都统称为组合, 因为它们在类中的表示方式相似。

2.10. Aggregation Between Same Class (同类间的聚合)

聚合可以存在于同一类的对象之间 (例如, 一个人可以有一个主管, 而主管也是人)。当一个人有多个主管时, 可以使用数组表示主管列表。

2.11. Example: The Course Class (示例: 课程类)

`Course` 类用于管理课程和学生。

- **数据字段**: `courseName` (字符串), `students` (字符串数组), `numberOfStudents` (整数)。
- **方法**: `addStudent`, `dropStudent`, `getStudents`, `getNumberOfStudents` 等。

2.12. Example: The StackOfIntegers Class (示例: 整数栈类)

`StackOfIntegers` 类用于存储整数的栈。

- **数据字段**: `elements` (整数数组), `size` (整数)。
- **方法**: `StackOfIntegers()` (构造函数), `empty()`, `peek()`, `push()`, `pop()`, `getSize()`。

2.13. Implementing StackOfIntegers Class (实现整数栈类)

栈的实现通常使用数组。

- `elements[capacity - 1]` 表示数组的顶部。
- `elements[size - 1]` 表示栈的当前顶部。
- `elements[0]` 表示栈的底部。

2.14. Wrapper Classes (包装类)

Java 为每种基本数据类型提供了对应的包装类。

- **包装类列表:** Boolean , Character , Byte , Short , Integer , Long , Float , Double 。
- **特点:**
 - i. 没有无参构造函数。
 - ii. 所有包装类的实例都是不可变的，即其内部值一旦创建就不能改变。

2.15. The Integer and Double Classes (Integer 和 Double 类)

- **构造函数:** 可以从基本数据类型值或表示数字的字符串构造。
- **类常量:** 如 MAX_VALUE 和 MIN_VALUE , 表示对应基本数据类型的最大/最小值。
- **转换方法:** 如 intValue() 、 doubleValue() 等，将包装对象转换为基本类型值。
- **静态 valueOf 方法:** 从字符串创建新的包装对象。
- **字符串解析方法:** 如 parseInt() 、 parseDouble() , 将数字字符串解析为基本类型值。

2.16. Automatic Conversion Between Primitive Types and Wrapper Class Types (基本类型与包装类类型之间的自动转换)

JDK 1.5 引入了自动装箱 (autoboxing) 和自动拆箱 (unboxing)，允许基本类型和包装类之间自动转换。

- **自动装箱:** 基本类型自动转换为包装类对象 (例如，int 转换为 Integer)。
- **自动拆箱:** 包装类对象自动转换为基本类型 (例如，Integer 转换为 int)。

2.17. BigInteger and BigDecimal (大整数和大数据)

- 用于处理非常大的整数 (BigInteger) 或高精度浮点值 (BigDecimal)。
- 它们都在 java.math 包中。
- 两者都是不可变的，并扩展了 Number 类，实现了 Comparable 接口。

2.18. The String Class (String 类)

- **构造字符串:** 可以使用字符串字面量或 new String() 。
- **常用操作:** 获取长度，检索单个字符，字符串连接 (concat)，子字符串 (substring)，比较 (equals , compareTo)，字符串转换，查找字符或子字符串，字符串与数组转换，字符与数字值转换为字符串。

2.19. Strings Are Immutable (字符串是不可变的)

`String` 对象是不可变的，其内容一旦创建就不能改变。

如果对 `String` 变量进行重新赋值，实际上是让该变量指向一个新的 `String` 对象，而不是修改原有对象。原有对象会变成垃圾等待回收。

2.20. Interned Strings (内部化字符串)

为了提高效率和节省内存，JVM 对字符串字面量使用唯一的实例（内部化）。

这意味着，如果使用字符串字面量创建两个内容相同的字符串，它们将引用内存中的同一个对象。

但是，如果使用 `new String()` 显式创建字符串对象，即使内容相同，也会创建新的对象。

2.21. Replacing and Splitting Strings (替换和分割字符串)

`java.lang.String` 类提供了替换和分割字符串的方法：

- `replace(oldChar, newChar)`：替换所有匹配字符。
- `replaceFirst(oldString, newString)`：替换第一个匹配子字符串。
- `replaceAll(oldString, newString)`：替换所有匹配子字符串。
- `split(delimiter)`：根据分隔符将字符串分割成数组。

2.22. Matching, Replacing and Splitting by Patterns (按模式匹配、替换和分割)

- 可以使用**正则表达式**来匹配、替换或分割字符串。
- `matches(regex)`：检查字符串是否匹配指定模式。
- `replaceAll(regex, replacement)`：使用正则表达式替换所有匹配项。
- `split(regex)`：使用正则表达式分割字符串。

2.23. Convert Character and Numbers to Strings (字符和数字转换为字符串)

`String` 类提供了多个静态 `valueOf` 方法，可以将字符、字符数组、数字（`double`、`long`、`int`、`float`）转换为字符串。

2.24. StringBuilder and StringBuffer (StringBuilder 和 StringBuffer)

- 是 `String` 类的替代品，用于处理**可变**字符串。
- `StringBuilder` 更灵活，允许添加、插入或追加新内容。

- `StringBuffer` 类似于 `StringBuilder`，但其方法是**同步**的，适用于多线程环境。

2.25. Modifying Strings in the Builder (在 `StringBuilder` 中修改字符串)

`StringBuilder` 提供了多种方法来修改字符串：

- `append()`：追加字符数组、基本类型值或字符串。
- `delete()` / `deleteCharAt()`：删除字符。
- `insert()`：在指定位置插入。
- `replace()`：替换子字符串。
- `reverse()`：反转字符串。
- `setCharAt()`：设置指定索引处的字符。

2.26. The `toString`, `capacity`, `length`, `setLength`, and `charAt` Methods (`toString`、`capacity`、`length`、`setLength` 和 `charAt` 方法)

`StringBuilder` 提供：

- `toString()`：返回 `String` 对象。
- `capacity()`：返回当前容量。
- `length()`：返回字符数。
- `setLength()`：设置新的长度。
- `charAt()`：返回指定索引处的字符。
- `substring()`：返回子字符串。
- `trimToSize()`：减少存储大小。

3. Chapter 11: Inheritance and Polymorphism (继承与多态)

3.1. Superclasses and Subclasses (超类和子类)

超类是更一般的类，子类是更具体的类。

子类可以从超类**继承**属性和方法。

3.2. Are superclass's Constructor Inherited? (超类的构造函数是否被继承?)

- 超类构造函数**不被继承**。
- 它们被**显式** (`super(...)`) 或**隐式** (`super()`) 调用。
- `super()` 必须是子类构造函数的第一条语句。

3.3. Constructor Chaining (构造函数链)

构造一个实例会调用继承链上所有超类的构造函数。

3.4. Defining a Subclass (定义子类)

子类可以：

- 添加新属性。
- 添加新方法。
- **重写**超类的方法。

3.5. Calling Superclass Methods (调用超类方法)

使用 `super.methodName()` 调用超类中的方法。

3.6. Overriding Methods in the Superclass (重写超类中的方法)

子类可以重写超类中已定义的方法的实现。

- **注意**：私有方法不能被重写。
- **注意**：静态方法不能被重写，如果子类重新定义了同名静态方法，它会**隐藏**超类的方法。

3.7. Overriding vs. Overloading (重写 vs. 重载)

- **重写 (Overriding)**：子类提供与超类方法**相同签名**的不同实现。
- **重载 (Overloading)**：在同一个类中定义**同名但参数列表不同**的方法。

3.8. The `Object` Class and Its Methods (Object 类及其方法)

- Java 中所有类都直接或间接继承自 `java.lang.Object` 。
- `Object` 类是所有类的根。
- **`toString()` 方法**：返回对象的字符串表示。建议重写此方法以提供有意义的输出。

3.9. Polymorphism (多态)

- 多态性意味着超类型变量可以引用子类型对象。
- **子类型**：由子类定义的类型。
- **超类型**：由超类定义的类型。

3.10. Dynamic Binding (动态绑定)

- **动态绑定**：Java 虚拟机（JVM）在运行时动态确定要调用哪个方法实现。
- 当一个超类型变量引用子类型对象并调用其方法时，JVM 会从最具体的类开始搜索方法实现，并调用找到的第一个。

3.11. Method Matching vs. Binding (方法匹配 vs. 绑定)

- **方法匹配**：编译器在编译时根据方法签名（参数类型、数量、顺序）查找方法。
- **方法绑定**：Java 虚拟机在运行时动态地将方法调用与实际的方法实现关联起来。

3.12. Generic Programming (泛型编程)

多态允许方法通用地处理各种对象参数。

3.13. Casting Objects (对象类型转换)

- **隐式类型转换 (Implicit Casting)**：从子类型到超类型是自动的，因为子类型对象总是超类型实例。
- **显式类型转换 (Explicit Casting)**：从超类型到子类型需要显式转换，因为超类型对象不一定是子类型实例。
- **注意**：显式类型转换可能失败（`ClassCastException`）。

3.14. The instanceof Operator (instanceof 运算符)

使用 `instanceof` 运算符测试一个对象是否是某个类的实例，以在进行向下转型前检查类型安全性。

3.15. The ArrayList Class (ArrayList 类)

- `ArrayList` 提供了可变大小的动态数组。
- 与固定大小的数组不同，`ArrayList` 可以根据需要增长或缩小。
- 提供了添加、删除、获取、设置元素等操作。

3.16. Generic Type (泛型类型)

`ArrayList` 是一个泛型类 (`ArrayList<E>`), 其中 `E` 是类型参数。
创建 `ArrayList` 时, 可以指定具体的类型, 如 `ArrayList<String>`。

3.17. Differences and Similarities between Arrays and ArrayList (数组和 ArrayList 的区别与相似之处)

- **数组**: 固定大小, 直接访问元素快。
- **ArrayList**: 动态大小, 增删元素方便, 但随机访问可能稍慢。

3.18. The MyStack Classes (MyStack 类)

基于 `ArrayList` 实现的栈, 提供 `push`、`pop`、`peek`、`isEmpty` 等栈操作。

3.19. The protected Modifier (protected 修饰符)

- `protected` 成员可以被同一包中的类和所有子类访问 (无论是否在同一包)。
- 可见性级别: `private < default < protected < public`。

3.20. Accessibility Summary (可访问性摘要)

修饰符	同一类	同一包	子类	不同包
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
default	✓	✓	✗	✗
private	✓	✗	✗	✗

3.21. The final Modifier (final 修饰符)

- **final 类**: 不能被继承。
- **final 变量**: 是常量, 值不能改变。
- **final 方法**: 不能被子类重写。

4. Chapter 12: Exception Handling and Text IO (异常处理和文本I/O)

4.1. Motivations (动机)

当程序运行时遇到运行时错误，程序会异常终止。如何处理运行时错误，使程序能够继续运行或优雅地终止？这就是本章将要介绍的主题。

4.2. Objectives (目标)

本章目标包括：概述异常和异常处理，探索使用异常处理的优势，区分异常类型（`Error` (致命) vs. `Exception` (非致命)，以及受检异常 (checked) vs. 非受检异常 (unchecked))，在方法头中声明异常，在方法中抛出异常，编写 `try-catch` 块来处理异常，解释异常如何传播，从异常对象中获取信息，开发带有异常处理的应用程序，在 `try-catch` 块中使用 `finally` 子句，仅对意外错误使用异常，在 `catch` 块中重新抛出异常，创建链式异常，定义自定义异常类，发现文件/目录属性，删除和重命名文件/目录，并使用 `File` 类创建目录，使用 `PrintWriter` 类将数据写入文件，使用 `try-with-resources` 确保资源自动关闭，使用 `Scanner` 类从文件中读取数据，理解如何使用 `Scanner` 读取数据，开发一个在文件中替换文本的程序，从 Web 读取数据，开发一个 Web 爬虫。

4.3. 异常处理

- **传统方法缺点：**代码量大，正常逻辑与异常处理混杂，依赖返回值表示错误状态。
- **异常处理优势：**简洁地处理异常，分离正常逻辑和异常处理逻辑。

4.4. Exception-Handling Overview (异常处理概述)

程序运行时可能会出现运行时错误，导致程序异常终止。异常处理机制允许程序继续运行或优雅地终止。

4.5. Exception Advantages (异常优势)

异常处理的优势在于它允许方法向其调用者抛出异常。若无此能力，方法必须自行处理异常或终止程序。

4.6. Handling `InputMismatchException` (处理 `InputMismatchException`)

通过处理 `InputMismatchException`，程序将持续读取输入，直到输入正确为止。

4.7. Exception Types (异常类型)

Java 异常层次结构：

- Object -> Throwable
 - Error (系统错误，通常无法恢复)
 - LinkageError
 - VirtualMachineError 等
 - Exception (程序错误或外部情况，可捕获和处理)
 - **受检异常 (Checked Exceptions)：**
 - ClassNotFoundException
 - IOException
 - **非受检异常 (Unchecked Exceptions)(RuntimeException 及其子类)：**
 - ArithmeticException
 - NullPointerException
 - IndexOutOfBoundsException
 - IllegalArgumentException

4.8. System Errors (系统错误)

系统错误由 JVM 抛出，并由 Error 类表示。Error 类描述内部系统错误。这些错误很少发生，如果发生，通常除了通知用户并尝试优雅地终止程序外，几乎无能为力。

4.9. Exceptions (异常)

Exception 描述由程序和外部情况引起的错误。这些错误可以被捕获和处理。

4.10. Runtime Exceptions (运行时异常)

RuntimeException 是由编程错误引起的，例如错误的类型转换、数组越界访问和数值错误。Java 不强制要求捕获非受检异常。

4.11. Checked Exceptions vs. Unchecked Exceptions (受检异常 vs. 非受检异常)

RuntimeException、Error 及其子类被称为**非受检异常**。所有其他异常被称为**受检异常**，这意味着编译器强制程序员检查和处理这些异常。

4.12. Unchecked Exceptions (非受检异常)

在大多数情况下，非受检异常反映了不可恢复的编程逻辑错误。例如，如果你在对象被赋值之前通过引用变量访问对象，就会抛出 `NullPointerException`；如果你访问数组中越界元素，就会抛出 `IndexOutOfBoundsException`。这些是应该在程序中纠正的逻辑错误。为了避免 `try-catch` 块的繁琐过度使用，Java 不强制编写代码来捕获非受检异常。

4.13. Declaring, Throwing, and Catching Exceptions (声明、抛出和捕获异常)

- **声明异常**：方法头使用 `throws` 关键字声明可能抛出的异常。
- **抛出异常**：程序检测到错误时，创建异常实例并用 `throw` 关键字抛出。
- **捕获异常**：使用 `try-catch` 块处理异常。

4.14. Declaring Exceptions (声明异常)

每个方法必须声明它可能抛出的受检异常类型。这被称为**声明异常**。

4.15. Throwing Exceptions (抛出异常)

当程序检测到错误时，程序可以创建适当的异常类型实例并抛出它。这被称为**抛出异常**。

4.16. Throwing Exceptions Example (抛出异常示例)

在 `setRadius` 方法中，如果 `newRadius` 小于 0，则抛出 `IllegalArgumentException`。

```
public void setRadius (double newRadius) throws IllegalArgumentException {  
    if (newRadius >= 0)  
        radius = newRadius;  
    else  
        throw new IllegalArgumentException("Radius cannot be negative");  
}
```

4.17. Catching Exceptions (捕获异常)

使用 `try-catch` 块捕获和处理异常。

```
try {
    statements; // 可能抛出异常的语句
} catch (Exception1 exVar1) {
    handler for exception1;
} catch (Exception2 exVar2) {
    handler for exception2;
} // ...
catch (ExceptionN exVar3) {
    handler for exceptionN;
}
```

4.18. Catch or Declare Checked Exceptions (捕获或声明受检异常)

Java 强制你处理受检异常。如果方法声明了一个受检异常（即除了 `Error` 或 `RuntimeException` 之外的异常），你必须在 `try-catch` 块中调用它，或者在调用方法中声明抛出该异常。

4.19. Example: Declaring, Throwing, and Catching Exceptions (示例：声明、抛出和捕获异常)

本示例演示了如何通过修改 `Circle` 类中的 `setRadius` 方法来声明、抛出和捕获异常，当半径为负数时抛出异常。

4.20. Rethrowing Exceptions (重新抛出异常)

在 `catch` 块中处理部分异常后，可以选择重新抛出异常，以便更高级别的调用者也能处理。

```
try {
    statements;
} catch (TheException ex) {
    perform operations before exits;
    throw ex; // 重新抛出异常
}
```

4.21. The `finally` Clause (`finally` 子句)

`finally` 块中的语句无论是否发生异常，都会被执行。它常用于清理资源，如关闭文件或网络连接。

```
try {
    statements;
} catch (TheException ex) {
    handling ex;
} finally {
    finalStatements; // 总是执行的语句
}
```

4.22. Trace a Program Execution (追踪程序执行)

程序执行的追踪演示了 try-catch-finally 块的行为：

- **无异常：** try 块执行 -> finally 块执行 -> 后续语句执行。
- **异常被捕获：** try 块中发生异常 -> 匹配的 catch 块处理 -> finally 块执行 -> 后续语句执行。
- **异常被重新抛出：** try 块中发生异常 -> 匹配的 catch 块处理并重新抛出 -> finally 块执行 -> 异常传播给调用者。

4.23. Cautions When Using Exceptions (使用异常时的注意事项)

异常处理将错误处理代码与正常编程任务分离，使程序更易读和维护。然而，异常处理通常需要更多时间和资源，因为它需要实例化新的异常对象、回滚调用栈以及将错误传播给调用方法。

4.24. When to Throw Exceptions (何时抛出异常)

当一个方法中发生异常，如果你希望该异常由其调用者处理，则应该创建并抛出异常。如果该异常可以在发生的方法中处理，则无需抛出。

4.25. When to Use Exceptions (何时使用异常)

try-catch 块应用于处理**意外错误条件**。不应将其用于处理简单、预期的情况。例如，检查引用变量是否为 null，使用 if (refVar != null) 比 try-catch (NullPointerException) 更高效。

4.26. Defining Custom Exception Classes (定义自定义异常类)

- 尽可能使用 API 中已有的异常类。
- 当预定义类不足以满足需求时，定义自定义异常类。
- 自定义异常类通过扩展 Exception 或 Exception 的子类来实现。

4.27. Custom Exception Class Example (自定义异常类示例)

在 `setRadius` 方法中，如果半径为负数，可以抛出自定义异常类（如 `InvalidRadiusException`）。

4.28. The File Class (File 类)

`File` 类旨在提供文件或路径的抽象，处理机器相关的复杂性，提供机器无关的方式。文件名是一个字符串。`File` 类是文件名称及其目录路径的包装类。

4.29. Obtaining file properties and manipulating file (获取文件属性和操作文件)

`java.io.File` 类提供了以下功能：

- **构造函数**：通过路径名、父目录和子文件路径创建 `File` 对象。
- **文件属性查询**：`exists()`、`canRead()`、`canWrite()`、`isDirectory()`、`isFile()`、`isAbsolute()`、`isHidden()`、`getAbsolutePath()`、`getCanonicalPath()`、`getName()`、`getPath()`、`getParent()`、`lastModified()`、`length()`。
- **文件操作**：`listFile()` (列出目录下的文件)、`delete()` (删除)、`renameTo()` (重命名)、`mkdir()` (创建目录)、`mkdirs()` (创建多级目录)。

4.30. Problem: Explore File Properties (问题：探索文件属性)

本节的目标是编写一个程序，演示如何以平台无关的方式创建文件，并使用 `File` 类的方法获取其属性。

4.31. Text I/O (文本I/O)

`File` 对象封装文件属性，但不包含读写数据方法。要执行 I/O 操作，需要使用相应的 Java I/O 类创建对象。本节介绍如何使用 `Scanner` 和 `PrintWriter` 类读写文本文件中的字符串和数值。

4.32. Writing Data Using PrintWriter (使用 PrintWriter 写入数据)

`java.io.PrintWriter` 类用于将数据写入文本文件。

- **构造函数**：根据文件名创建 `PrintWriter` 对象。
- **写入方法**：`print()` 和 `println()` 系列方法，可写入字符串、字符、各种基本数据类型和字符数组。
- `println` 会在写入后添加行分隔符（在 Windows 上是 `\r\n`，在 Unix 上是 `\n`）。

- 也支持 `printf` 格式化输出。

4.33. Try-with-resources (try-with-resources)

JDK 7 引入了 `try-with-resources` 语法，确保资源（如文件）在块结束时自动关闭，避免程序员忘记关闭文件。

```
try (declare and create resources) {  
    Use the resource to process the file;  
}
```

4.34. Reading Data Using Scanner (使用 Scanner 读取数据)

`java.util.Scanner` 类用于从文件或字符串中读取数据。

- **构造函数**：可以从 `File` 或 `String` 创建 `Scanner` 对象。
- **关闭**：`close()` 关闭此扫描器。
- **读取方法**：`hasNext()` (检查是否有下一个 token)、`next()` (返回下一个 token，作为字符串)、`nextByte()`、`nextShort()`、`nextInt()`、`nextLong()`、`nextFloat()`、`nextDouble()`。
- `useDelimiter()`：设置扫描器的分隔模式。

4.35. Problem: Replacing Text (问题：替换文本)

本节的目标是编写一个名为 `ReplaceText` 的类，用新字符串替换文本文件中的字符串。文件名和字符串作为命令行参数传入。

4.36. Reading Data from the Web (从 Web 读取数据)

可以像从本地文件读取数据一样，从 Web 读取数据。

通过 `URL` 对象和 `openStream()` 方法获取输入流，然后用 `Scanner` 读取数据。

4.37. Case Study: Web Crawler (案例研究：网络爬虫)

本案例研究开发一个通过跟踪超链接来遍历 Web 的程序。为了避免重复访问 URL，程序维护两个 URL 列表：一个存储待遍历的 URL，另一个存储已遍历的 URL。

5. Chapter 13: Abstract Classes and Interfaces (抽象类和接口)

5.1. Motivations (动机)

在开发 GUI 程序时，需要编写代码来响应用户操作。这需要了解接口，接口用于定义类的共同行为。在深入接口之前，本章将介绍抽象类。

5.2. Objectives (目标)

本章目标包括：设计和使用抽象类，使用抽象 `Number` 类泛化数值包装类，使用 `Calendar` 和 `GregorianCalendar` 处理日历，使用接口指定对象共同行为，实现 `Comparable` 接口定义自然顺序，使用 `Cloneable` 接口进行对象克隆，探讨具体类、抽象类和接口异同，设计 `Rational` 类处理有理数，以及遵循类设计指南。

5.3. Abstract Classes and Abstract Methods (抽象类和抽象方法)

- **抽象类**：用 `abstract` 关键字修饰的类。抽象类不能被实例化，但可以作为其他类的父类。
- **抽象方法**：用 `abstract` 关键字修饰且没有方法体的方法。抽象方法只能在抽象类中声明。
- UML 图中，抽象类名和抽象方法用斜体表示。

5.4. Abstract method in abstract class (抽象类中的抽象方法)

抽象方法不能包含在非抽象类中。如果抽象超类的子类没有实现所有的抽象方法，那么子类也必须被定义为抽象类。换句话说，从抽象类继承的非抽象子类，必须实现所有抽象方法，即使这些方法在子类中没有被直接使用。

5.5. Object cannot be created from abstract class (抽象类不能创建对象)

抽象类不能使用 `new` 运算符实例化。然而，你可以定义其构造函数，这些构造函数会在其子类的构造函数中被调用。

5.6. Abstract class without abstract method (没有抽象方法的抽象类)

可以定义一个不包含任何抽象方法的抽象类。在这种情况下，你不能使用 `new` 运算符创建该类的实例。这个类被用作定义新子类的基类。

5.7. Superclass of abstract class may be concrete (抽象类的超类可以是具体类)

一个子类可以是抽象的，即使它的超类是具体的。例如，`Object` 类是具体的，但它的子类（如 `GeometricObject`）可以是抽象的。

5.8. Concrete method overridden to be abstract (具体方法被重写为抽象方法)

子类可以重写其超类中的具体方法并将其定义为抽象方法。这种情况比较罕见，但当超类中的方法实现对子类变得无效时会很有用。在这种情况下，子类必须被定义为抽象类。

5.9. Abstract class as type (抽象类作为类型)

你不能从抽象类创建实例，但抽象类可以作为数据类型使用。因此，创建元素类型为 `GeometricObject` 的数组是正确的：`GeometricObject[] geo = new GeometricObject[10];`

5.10. Case Study: the Abstract Number Class (案例研究：抽象 Number 类)

`java.lang.Number` 是一个抽象类，是所有数值包装类（`Integer`，`Double`，`Long`，`Float`，`Short`，`Byte`，`BigInteger`，`BigDecimal`）的超类。它提供了将数值转换为各种基本数据类型（如 `byteValue()`，`shortValue()`，`intValue()`，`longValue()`，`floatValue()`，`doubleValue()`）的抽象方法。

5.11. The Abstract Calendar Class and Its GregorianCalendar subclass (抽象 Calendar 类及其 GregorianCalendar 子类)

`java.util.Date` 表示毫秒精度的时间瞬间。`java.util.Calendar` 是一个抽象基类，用于提取日期和时间信息（年、月、日、小时、分钟、秒）。`Calendar` 的子类可以实现特定的日历系统，如公历（`GregorianCalendar`）。`java.util.GregorianCalendar` 是 Java API 中支持的公历实现。

5.12. The get Method in Calendar Class (Calendar 类中的 get 方法)

`Calendar` 类中的 `get(int field)` 方法用于提取日期和时间信息。`field` 参数是常量，表示要获取的日期/时间字段，如 `YEAR`，`MONTH`，`DATE`，`HOUR`，`MINUTE`，`SECOND` 等。

5.13. Interfaces (接口)

- **什么是接口?** : 一个类似于类的构造, 只包含常量和抽象方法。
- **为什么接口有用?** : 它类似于抽象类, 但其目的是指定对象的行为。
- **如何定义接口?** : 使用

```
public interface InterfaceName { constant declarations; abstract method signatures; }
```

- **如何使用接口?** : 通过类实现接口。

5.14. What is an interface? Why is an interface useful? (什么是接口? 为什么接口有用?)

接口是类似于类的构造, 只包含常量和抽象方法。它类似于抽象类, 但其目的是指定对象的行为, 即使这些类之间没有直接的继承关系。

5.15. Define an Interface (定义接口)

使用 `public interface InterfaceName { ... }` 语法定义接口。接口可以包含常量声明和抽象方法签名。

```
public interface Edible {  
    /** Describe how to eat */  
    public abstract String howToEat();  
}
```

5.16. Interface is a Special Class (接口是一种特殊类)

接口被视为 Java 中的特殊类。每个接口都被编译成独立的字节码文件。与抽象类类似, 你不能使用 `new` 运算符创建接口的实例。但在大多数情况下, 你可以像使用抽象类一样使用接口, 例如, 作为变量的数据类型或类型转换的结果。

5.17. Omitting Modifiers in Interfaces (接口中省略修饰符)

接口中所有数据字段默认都是 `public final static`, 所有方法默认都是 `public abstract`。因此, 这些修饰符可以省略。接口中定义的常量可以通过 `InterfaceName.CONSTANT_NAME` 语法访问。

5.18. Example: The Comparable Interface (示例：Comparable 接口)

`java.lang.Comparable<E>` 接口定义了对对象的自然排序。它包含一个抽象方法：`public int compareTo(E o);` `compareTo` 方法返回负整数、零或正整数，分别表示当前对象小于、等于或大于指定对象。

5.19. The toString , equals , and hashCode Methods (toString、equals 和 hashCode 方法)

每个包装类都重写了 `Object` 类中的 `toString`、`equals` 和 `hashCode` 方法。所有数值包装类和 `Character` 类都实现了 `Comparable` 接口，因此它们也实现了 `compareTo` 方法。

5.20. Integer and BigInteger Classes (Integer 和 BigInteger 类)

`Integer` 和 `BigInteger` 类都实现了 `Comparable` 接口，以便它们的对象可以进行比较。

5.21. The Cloneable Interfaces (Cloneable 接口)

- **标记接口 (Marker Interface):** `Cloneable` 是一个空接口，不包含常量或方法。
- 它的作用是标记一个类拥有某种期望的属性，即其对象可以被克隆。
- 实现了 `Cloneable` 接口的类的对象可以使用 `Object` 类中定义的 `clone()` 方法进行克隆。

5.22. Shallow vs. Deep Copy (浅拷贝 vs. 深拷贝)

- **浅拷贝 (Shallow Copy):** 只复制对象本身和其基本类型字段的值。如果对象包含引用类型字段，则只复制引用，不复制引用的对象。这意味着新对象和原对象会共享引用的子对象。
- **深拷贝 (Deep Copy):** 不仅复制对象本身，还递归地复制其引用的所有子对象，确保新对象与原对象完全独立。

5.23. Interfaces vs. Abstract Classes (接口 vs. 抽象类)

特性	抽象类 (Abstract class)	接口 (Interface)
变量	无限制（可以有各种类型）	必须是 <code>public static final</code> 常量
构造函数	被子类通过构造函数链调用。不能直接实例化。	没有构造函数。不能实例化。
方法	无限制（可以有具体方法、抽象方法）	必须是 <code>public abstract</code> 实例方法

5.24. Interfaces vs. Abstract Classes, cont. (接口 vs. 抽象类, 续)

所有类都共享一个根——Object 类，但接口没有单一的根。接口也可以定义一个类型。接口类型的变量可以引用实现该接口的类的任何实例。如果类扩展一个接口，这个接口扮演着超类的角色。可以将接口用作数据类型，并进行类型转换。

5.25. Caution: conflict interfaces (注意：冲突接口)

在极少数情况下，一个类可能实现两个存在冲突信息的接口（例如，两个具有不同值的同名常量，或两个签名相同但返回类型不同的方法）。这种类型的错误将被编译器检测到。

5.26. Whether to use an interface or a class? (何时使用接口或类?)

- **抽象类和接口**都可以用于建模共同特征。
- **强“is-a”关系（父子关系）应使用类**来建模（例如，员工是人）。
- **弱“is-a”关系（一种属性）应使用接口**来建模（例如，所有字符串都是可比较的，所以 String 类实现 Comparable 接口）。
- 接口还可用于**规避单继承限制**，实现多重继承的效果。

5.27. The Rational Class (有理数类)

Rational 类（有理数）的 UML 图和方法，它实现了 java.lang.Comparable 接口并扩展了 java.lang.Number 抽象类。提供了有理数的算术运算（加、减、乘、除）以及获取分子、分母和字符串表示的方法。

5.28. Designing a Class (设计类)

- **内聚性 (Coherence)**：一个类应描述一个单一实体，所有操作应逻辑地支持一个内聚目的。
- **分离职责 (Separating responsibilities)**：职责过多的实体可拆分为多个类。例如，String、StringBuilder、StringBuffer 分别处理不可变、可变（非同步）和可变（同步）字符串。
- **重用性**：设计类时应考虑其不同组合、顺序和环境下的重用性。属性应可按任意顺序设置，方法应独立于其调用顺序。
- 提供**公共无参构造函数**，并尽可能重写 Object 类中的 equals 和 toString 方法。
- 遵循标准 Java 编程风格和命名约定。选择信息丰富的名称。
- 始终将数据声明放在构造函数之前，构造函数放在方法之前。
- 始终提供构造函数并初始化变量，以避免编程错误。

5.29. Using Visibility Modifiers (使用可见性修饰符)

- 类可以呈现两种契约：一种面向用户，一种面向扩展者。
- 数据字段应为 `private`，访问器方法（`get`）应为 `public`，供用户访问。
- 数据字段或方法若意在供扩展者使用，可设为 `protected`。
- 扩展者可以提高实例方法的可见性（如从 `protected` 到 `public`），或改变其实现，但不应以违反契约的方式改变。

5.30. Using the `static` Modifier (使用 `static` 修饰符)

由类的所有实例共享的属性应声明为 `static` 属性。

6. Chapter 14: JavaFX Basics (JavaFX 基础)

6.1. Motivations (动机)

JavaFX 是一个用于开发 Java GUI 程序的新框架。它展示了面向对象原则的优秀应用。本章旨在介绍 JavaFX 编程基础，并演示如何用 JavaFX 实现 OOP。具体而言，本章将介绍 JavaFX 框架以及 GUI 组件及其关系。

6.2. Objectives (目标)

本章目标包括：区分 JavaFX、Swing 和 AWT，编写简单的 JavaFX 程序并理解舞台 (stage)、场景 (scene) 和节点 (node) 之间的关系，使用窗格 (panes)、UI 控件和形状 (shapes) 创建用户界面，使用绑定属性 (binding properties) 同步属性值，使用通用属性 `style` 和 `rotate` 进行节点操作，使用 `Color` 类创建颜色，使用 `Font` 类创建字体，使用 `Image` 和 `ImageView` 类创建和显示图像，使用 `Pane`、`StackPane`、`FlowPane`、`GridPane`、`BorderPane`、`HBox` 和 `VBox` 布局节点，使用 `Text` 类显示文本并使用 `Line`、`Circle`、`Rectangle`、`Ellipse`、`Arc`、`Polygon` 和 `Polyline` 创建形状，以及开发可重用 GUI 组件 `ClockPane` 以显示模拟时钟。

6.3. JavaFX vs Swing and AWT (JavaFX 与 Swing 和 AWT 对比)

JavaFX 旨在取代 Swing 和 AWT，用于开发丰富的互联网应用程序。

- **AWT (Abstract Windows Toolkit)**：Java 早期 GUI 库，适用于简单 GUI，但易出现平台特定 bug。
- **Swing**：取代 AWT 的更健壮、多功能和灵活的库，组件直接绘制在画布上，对平台依赖性较低。

- **JavaFX**: Java 8 引入的全新 GUI 平台。

6.4. 创建 JavaFX 项目 (Creating JavaFX Projects)

可以通过以下方式创建 JavaFX 项目：

- Maven：使用 Maven archetype。
- VSCode 插件：
 - i. Extension Pack for Java (microsoft)
 - ii. Maven for Java
 - iii. Project Manager for Java
 - iv. Language Support for Java(TM) by Red Hat

6.5. Basic Structure of JavaFX (JavaFX 基本结构)

JavaFX 应用程序的基本结构包括：

- **Application (应用程序)**: JavaFX 程序的入口点。
- **Override the start(Stage) method (重写 start(Stage) 方法)**: 应用程序的主入口点。
- **Stage (舞台)**: 顶级容器，代表窗口。
- **Scene (场景)**: 舞台中的内容容器，包含所有 UI 节点。
- **Nodes (节点)**: 场景中的图形元素（如按钮、文本等）。

6.6. Panes, UI Controls, and Shapes (窗格、UI 控件和形状)

- **节点 (Node)**: 所有可见元素（包括 UI 控件、形状等）的基类。
- **父节点 (Parent)**: 可以包含其他节点的节点，如 Pane 和 Control。
- **窗格 (Pane)**: 用于布局节点的容器，包括
FlowPane、GridPane、BorderPane、HBox、VBox、StackPane。
- **UI 控件 (Control)**: 用户界面交互元素，如
Label、TextField、Button、CheckBox、RadioButton、TextArea。
- **形状 (Shape)**: 图形元素，如
Line、Circle、Ellipse、Rectangle、Path、Polygon、Polyline、Text。
- **ImageView**: 用于显示图像。

6.7. Display a Shape (显示形状)

JavaFX 使用传统的数学坐标系，y 轴向下。

6.8. Binding Properties (绑定属性)

JavaFX 引入了**绑定属性**的概念。它允许一个目标对象（或其属性）与一个源对象（或其属性）绑定。当源对象的值发生变化时，目标属性会自动更新。目标对象被称为**绑定对象**或**绑定属性**。

6.9. Uni/Bidirectional Binding (单向/双向绑定)

- **单向绑定**：源属性的变化影响目标属性，但目标属性的变化不影响源属性。
- **双向绑定**：源属性和目标属性相互影响，任何一方的变化都会同步到另一方。

6.10. Common Properties and Methods for Nodes (节点的通用属性和方法)

- **style**：用于设置 JavaFX CSS 样式。
- **rotate**：用于旋转节点。

6.11. The Color Class (Color 类)

`javafx.scene.paint.Color` 类用于表示颜色。

- **属性**：`red`、`green`、`blue` (0.0-1.0 范围) 和 `opacity` (透明度, 0.0-1.0 范围)。
- **构造函数**：根据 RGB 和透明度值创建颜色。
- **方法**：`brighter()` (更亮)、`darker()` (更暗)，以及获取不透明或 0-255 RGB 值的颜色。

6.12. The Font Class (Font 类)

`javafx.scene.text.Font` 类用于管理字体。

- **属性**：`size`、`name` 和 `family`。
- **构造函数**：可指定字体名称、大小、粗细、姿态等。
- **方法**：`getFamilies()` (获取字体家族列表)、`getFontNames()` (获取完整字体名称列表)。

6.13. The Image Class (Image 类)

`javafx.scene.image.Image` 类用于加载图像。

- **属性**：`error` (加载是否正确)、`height`、`width` (图像高度和宽度)、`progress` (加载进度)。
- **构造函数**：通过文件名或 URL 创建 `Image` 对象。

6.14. The ImageView Class (ImageView 类)

`javafx.scene.image.ImageView` 类是 `Node` 的子类，用于显示 `Image` 对象。

- **属性：** `fitHeight`、`fitWidth` (适应高度和宽度)、`x`、`y` (图像视图原点坐标)、`image` (要显示的图像)。
- **构造函数：** 创建空的 `ImageView`、或带指定图像的 `ImageView`、或从文件/URL 加载图像的 `ImageView`。

6.15. Layout Panes (布局窗格)

JavaFX 提供多种窗格来组织容器中的节点。

- **Pane**：布局窗格的基类，包含 `getChildren()` 方法。
- **StackPane**：节点堆叠在彼此上方，居中显示。
- **FlowPane**：节点按行或列垂直/水平排列。
- **GridPane**：节点放置在二维网格的单元格中。
- **BorderPane**：节点放置在顶部、右侧、底部、左侧和中心区域。
- **HBox**：节点在一行中放置。
- **VBox**：节点在一列中放置。

6.16. FlowPane (流式窗格)

`FlowPane` 用于按行或列布局节点。

- **对齐方式 (alignment)**：默认 `Pos.LEFT`。
- **方向 (orientation)**：默认 `Orientation.HORIZONTAL`。
- **水平间距 (hgap)** 和 **垂直间距 (vgap)**。

6.17. GridPane (网格窗格)

`GridPane` 用于将节点放置在二维网格的单元格中。

- **对齐方式 (alignment)**：默认 `Pos.LEFT`。
- **网格线可见性 (gridLinesVisible)**：默认 `false`。
- **水平间距 (hgap)** 和 **垂直间距 (vgap)**。
- **添加节点 (add)**：指定子节点、列索引和行索引。

6.18. BorderPane (边框窗格)

`BorderPane` 用于将节点放置在顶部、右侧、底部、左侧和中心区域。

- 每个区域只能放置一个节点。
- 提供了设置每个区域节点和对齐方式的方法。

6.19. HBox (水平箱式窗格)

HBox 用于将节点排列在单行中。

- **对齐方式 (alignment)**: 默认 Pos.TOP_LEFT 。
- **填充高度 (fillHeight)**: 默认 true ，可调整子节点填充高度。
- **间距 (spacing)**: 子节点之间的水平间距。

6.20. VBox (垂直箱式窗格)

VBox 用于将节点排列在单列中。

- **对齐方式 (alignment)**: 默认 Pos.TOP_LEFT 。
- **填充宽度 (fillWidth)**: 默认 true ，可调整子节点填充宽度。
- **间距 (spacing)**: 子节点之间的垂直间距。

6.21. Shapes (形状)

JavaFX 提供多种形状类，用于绘制文本、线条、圆形、矩形、椭圆、弧、多边形和多段线。这些形状类都继承自 Node 。

6.22. Text (文本)

javafx.scene.text.Text 类用于显示文本。

- **属性**: text (要显示的文本)、 x 、 y (文本坐标)、 underline (下划线)、 strikethrough (删除线)、 font (字体)。
- **构造函数**: 创建空文本、或带指定文本的文本、或带指定坐标和文本的文本。

6.23. Line (线条)

javafx.scene.shape.Line 类用于绘制线条。

- **属性**: startX 、 startY (起点坐标)、 endX 、 endY (终点坐标)。
- **构造函数**: 创建空线条或带指定起终点坐标的线条。

6.24. Rectangle (矩形)

`javafx.scene.shape.Rectangle` 类用于绘制矩形。

- **属性：** `x`、`y` (左上角坐标)、`width`、`height` (宽度和高度)、`arcWidth`、`arcHeight` (圆弧宽度和高度，用于圆角矩形)。
- **构造函数：** 创建空矩形或带指定属性的矩形。

6.25. Circle (圆形)

`javafx.scene.shape.Circle` 类用于绘制圆形。

- **属性：** `centerX`、`centerY` (圆心坐标)、`radius` (半径)。
- **构造函数：** 创建空圆形或带指定圆心和半径的圆形。

6.26. Ellipse (椭圆)

`javafx.scene.shape.Ellipse` 类用于绘制椭圆。

- **属性：** `centerX`、`centerY` (圆心坐标)、`radiusX`、`radiusY` (水平和垂直半径)。
- **构造函数：** 创建空椭圆或带指定属性的椭圆。

6.27. Arc (弧)

`javafx.scene.shape.Arc` 类用于绘制弧。

- **属性：** `centerX`、`centerY` (圆心坐标)、`radiusX`、`radiusY` (水平和垂直半径)、`startAngle` (起始角度)、`length` (弧长)、`type` (闭合类型，如 `OPEN`，`CHORD`，`ROUND`)。
- **构造函数：** 创建空弧或带指定属性的弧。

6.28. Polygon and Polyline (多边形和多段线)

- **Polygon (多边形)：** 由一系列连接点组成的闭合形状。
- **Polyline (多段线)：** 由一系列连接点组成的非闭合形状。
- 两者都通过 `getPoints()` 返回一个 `ObservableList<Double>`，存储点的 `x` 和 `y` 坐标。

6.29. Case Study: The ClockPane Class (案例研究：时钟窗格类)

`ClockPane` 类是一个自定义窗格，用于显示模拟时钟。

- **属性：** `hour`、`minute`、`second` (时、分、秒)。

- **方法：** `ClockPane()` (构造函数)、`setCurrentTime()` (设置当前时间)、`setWidth()`、`setHeightTime()` (设置宽度和高度并重绘时钟)。

7. Chapter 15: Event-Driven Programming and Animations (事件驱动编程和动画)

7.1. Motivations (动机)

开发 GUI 程序需要处理用户交互。**事件驱动编程**是一种响应用户操作（如点击按钮）的编程范式。

7.2. Objectives (目标)

本章目标包括：理解事件驱动编程，描述事件、事件源和事件类，定义处理类、注册处理对象和编写事件处理代码，使用内部类和匿名内部类定义处理类，使用 Lambda 表达式简化事件处理，开发贷款计算器 GUI，处理鼠标事件 (`MouseEvent`) 和键盘事件 (`KeyEvent`)，创建监听器处理可观察对象的值变化，使用 `Animation`、`PathTransition`、`FadeTransition` 和 `Timeline` 类开发动画，以及模拟弹跳球动画。

7.3. Procedural vs. Event-Driven Programming (过程式 vs. 事件驱动编程)

- **过程式编程：** 代码按程序顺序执行。
- **事件驱动编程：** 代码在事件激活时执行。

7.4. Taste of Event-Driven Programming (事件驱动编程初体验)

一个简单的 GUI 示例，显示一个按钮。当按钮被点击时，控制台会显示消息，演示事件响应。

7.5. Handling GUI Events (处理 GUI 事件)

- **事件源对象 (Source object)：** 生成事件的对象（例如，按钮）。
- **监听器对象 (Listener object)：** 包含处理事件的方法。
- **事件：** 由事件源对象触发。
- **处理程序 (Handler)：** 监听器对象中的方法，实际处理事件。

7.6. Trace Execution (追踪执行)

一个 JavaFX 事件处理的追踪示例：

1. 从 `main` 方法开始，创建 GUI 窗口。
2. 点击“OK”按钮。
3. JVM 调用监听器的 `handle` 方法。
4. `handle` 方法中的代码执行，例如在控制台打印消息。

7.7. Events (事件)

- 事件可以定义为程序收到的一种信号，表明发生了某事。
- 事件通常由外部用户操作生成，例如鼠标移动、鼠标点击或键盘按键。

7.8. Event Classes (事件类)

JavaFX 事件类位于 `javafx.event` 包中。

事件类层次结构：

- `EventObject` (根) -> `Event`
 - `ActionEvent` (按钮点击等)
 - `InputEvent` (输入事件)
 - `MouseEvent` (鼠标事件)
 - `KeyEvent` (键盘事件)
 - `WindowEvent` (窗口事件)

7.9. Event Information (事件信息)

- 事件对象包含与事件相关的所有属性。
- 可以使用 `EventObject` 类中的 `getSource()` 方法识别事件源对象。
- `EventObject` 的子类处理特殊类型的事件，如按钮动作、窗口事件、鼠标移动和按键。

7.10. Selected User Actions and Handlers (选定的用户操作和处理程序)

表格列出了常见用户操作、事件源对象、事件类型以及相应的事件注册方法。

例如，按钮点击触发 `ActionEvent`，注册方法为 `setOnAction(EventHandler<ActionEvent>)`。鼠标事件和键盘事件则有更细分的类型和注册方法。

7.11. The Delegation Model (委托模型)

- **事件源**：生成事件并注册监听器。
- **监听器**：实现事件处理接口，包含处理事件的 `handle` 方法。
- **注册**：事件源将监听器对象注册到自身，以便在事件发生时调用 `handle` 方法。

7.12. The Delegation Model: Example (委托模型：示例)

示例代码展示了如何创建按钮 (`Button`)、处理类 (`OKHandlerClass`) 和事件注册 (`setOnAction`)。

```
Button btOK = new Button("OK"); //
OKHandlerClass handler = new OKHandlerClass(); //
btOK.setOnAction(handler); // 注册事件处理
```

7.13. Example: First Version for ControlCircle (no listeners) (示例：ControlCircle 的第一个版本（无监听器）)

演示一个包含两个按钮（放大、缩小）控制圆形大小的程序，但尚未添加监听器，因此按钮无效。

7.14. Example: Second Version for ControlCircle (with listener for Enlarge) (示例：ControlCircle 的第二个版本（带放大监听器）)

在第一个版本的基础上，为“放大”按钮添加了监听器，使其能够响应点击事件并改变圆形大小。

7.15. Inner Class Listeners (内部类监听器)

- 监听器类通常只用于特定 GUI 组件，不共享给其他应用程序。
- 适合将监听器类定义为框架类内部的**内部类 (inner class)**。

7.16. Inner Classes (内部类)

- **定义**：一个类是另一个类的成员。
- **优势**：
 - 使程序更简单。
 - 内部类可以访问其包含的外部类的数据和方法，无需传递外部类引用给内部类构造函数。

7.17. Inner Classes, cont. (内部类，续)

内部类使得程序更简洁。

内部类被编译成 `OuterClassName$InnerClassName.class` 的形式。

内部类可以声明为 `public`、`protected` 或 `private`，遵循与成员相同的可见性规则。

静态内部类可以不依赖外部类实例而访问，但不能访问外部类的非静态成员。

7.18. Anonymous Inner Classes (匿名内部类)

- **定义：**没有名称的内部类，通常直接在创建实例时声明。
- **特性：**
 - 必须扩展一个超类或实现一个接口，但不能有显式的 `extends` 或 `implements` 子句。
 - 必须实现超类或接口中的所有抽象方法。
 - 总是使用其超类的无参构造函数创建实例（如果实现接口，则使用 `Object()` 的无参构造函数）。
 - 编译后名为 `OuterClassName$n.class`，其中 `n` 是数字。

7.19. Simplifying Event Handling Using Lambda Expressions (使用 Lambda 表达式简化事件处理)

Lambda 表达式是 Java 8 的新特性，可将匿名方法简化为更简洁的语法。它们是**单抽象方法接口 (SAM)** 的实现，即接口中只有一个抽象方法。

7.20. Basic Syntax for a Lambda Expression (Lambda 表达式基本语法)

- `(type1 param1, type2 param2, ...) -> expression`
- `(type1 param1, type2 param2, ...) -> { statements; }`
- 参数的数据类型可以显式声明或隐式推断。如果只有一个参数且数据类型可推断，则括号可以省略。

7.21. The MouseEvent Class (MouseEvent 类)

`javafx.scene.input.MouseEvent` 类用于处理鼠标事件。

- **信息：**按钮（左/中/右）、点击次数、`x/y` 坐标（事件源/场景/屏幕）、是否按下了 `Alt/Control/Meta/Shift` 键。

7.22. The KeyEvent Class (KeyEvent 类)

`javafx.scene.input.KeyEvent` 类用于处理键盘事件。

- **信息：** 字符 (`getCharacter()`)、 键码 (`getCode()`)、 文本 (`getText()`)、 是否按下了 `Alt/Control/Meta/Shift` 键。

7.23. The KeyCode Constants (KeyCode 常量)

`KeyCode` 类定义了键盘按键的常量，例如 `HOME`，`END`，`PAGE_UP`，`ENTER`，`F1` 到 `F12`，`0` 到 `9`，`A` 到 `Z` 等。

7.24. Listeners for Observable Objects (可观察对象的监听器)

- 可以为**可观察对象 (Observable object)** 添加监听器，以处理其值的变化。
- 可观察对象具有 `addListener(InvalidationListener listener)` 方法。
- 监听器类应实现 `InvalidationListener` 接口，并使用 `invalidated(Observable o)` 方法处理属性值变化。

7.25. Animation (动画)

JavaFX 提供了 `Animation` 类作为所有动画的核心功能。

- **属性：** `autoReverse` (是否反向播放)、`cycleCount` (循环次数)、`rate` (播放速度)、`status` (动画状态)。
- **方法：** `pause()` (暂停)、`play()` (播放)、`stop()` (停止并重置)。

7.26. PathTransition (路径过渡)

`PathTransition` 类动画化一个节点沿着给定路径移动。

- **属性：** `duration` (持续时间)、`node` (目标节点)、`orientation` (节点方向)、`path` (形状路径)。
- **构造函数：** 可指定持续时间、路径和节点。

7.27. FadeTransition (淡入淡出过渡)

`FadeTransition` 类动画化节点透明度的变化。

- **属性：** `duration` (持续时间)、`node` (目标节点)、`fromValue` (起始透明度)、`toValue` (结束透明度)、`byValue` (透明度增量)。
- **构造函数：** 可指定持续时间、节点和透明度值。

7.28. Timeline (时间轴)

`Timeline` 类用于程序化地创建动画，通过一个或多个 `KeyFrame` 来实现。每个 `KeyFrame` 在指定的时间间隔内顺序执行。`Timeline` 继承自 `Animation`。

7.29. Case Study: Bouncing Ball (案例研究：弹跳球)

通过 `BallPane` 类表示一个弹跳球，并通过 `BounceBallControl` 类控制其运动，可以使用滑块控制速度。

8. Chapter 16: JavaFX UI Controls and Multimedia (JavaFX UI 控件和多媒体)

8.1. Motivations (动机)

图形用户界面 (GUI) 使系统用户友好且易于使用。创建 GUI 需要创造力和 GUI 组件的工作原理知识。Java 中的 GUI 组件非常灵活和多功能，可以创建各种有用的用户界面。

8.2. Objectives (目标)

本章目标包括：使用各种 UI 控件创建 GUI，使用 `Label` 及其属性，使用 `Button` 及其 `setOnAction`，使用 `CheckBox` 和 `RadioButton`（配合 `ToggleGroup`），使用 `TextField` 和 `PasswordField` 输入数据，使用 `TextArea` 输入多行数据，使用 `ComboBox` 单选，使用 `ListView` 单选/多选，使用 `ScrollBar` 和 `Slider` 选择范围，开发 Tic-Tac-Toe 游戏，使用 `Media`、`MediaPlayer` 和 `MediaView` 播放视频和音频，以及开发显示国旗和播放国歌的案例研究。

8.3. Frequently Used UI Controls (常用 UI 控件)

JavaFX 中常用的 UI 控件包括：`Label`，`Button`，`CheckBox`，`RadioButton`，`TextField`，`PasswordField`，`TextArea`，`ComboBox`，`ListView`，`ScrollBar`，`Slider`。这些控件通常继承自 `Control` 或 `Labeled`。

8.4. Labeled (带标签的控件)

`Labeled` 类是显示短文本、节点或两者皆有的显示区域的基类。`Label` 和 `Button` 等控件共享 `Labeled` 中定义的许多通用属性。

- **属性：** alignment (对齐方式)、 contentDisplay (内容显示方式)、 graphic (图形)、 graphicTextGap (图形与文本间距)、 textFill (文本颜色)、 text (文本内容)、 underline (下划线)、 wrapText (文本换行)。

8.5. Label (标签)

Label 类定义了标签。

- **构造函数：** 创建空标签、带文本的标签、或带文本和图形的标签。

8.6. ButtonBase and Button (按钮基类和按钮)

- **ButtonBase：** 是触发动作事件的控件的基类。
- **Button：** 触发点击事件的按钮。
- **共同特性：** onAction (动作事件处理器)。
- **构造函数：** 创建空按钮，或带文本的按钮，或带文本和图形的按钮。

8.7. CheckBox (复选框)

CheckBox 用于用户进行选择。它继承了 ButtonBase 和 Labeled 的所有属性（如 onAction、text、graphic、alignment 等）。

- **属性：** selected (是否选中)。
- **构造函数：** 创建空复选框或带文本的复选框。

8.8. RadioButton (单选按钮)

RadioButton 允许用户从一组选项中选择单个项目。它继承了 ToggleButton 的所有属性。

- **外观：** 通常显示为一个圆形，选中时填充。
- **属性：** selected (是否选中)、 toggleGroup (所属的单选组)。

8.9. TextField (文本字段)

TextField 用于输入或显示单行字符串。它是 TextInputControl 的子类。

- **属性：** text (文本内容)、 editable (是否可编辑)、 alignment (文本对齐)、 prefColumnCount (首选列数)、 onAction (动作事件处理器)。
- **构造函数：** 创建空文本字段或带初始文本的文本字段。

8.10. TextArea (文本区域)

TextArea 允许用户输入多行文本。

- **属性：** `prefColumnCount` (首选列数)、 `prefRowCount` (首选行数)、 `wrapText` (是否换行)。
- **构造函数：** 创建空文本区域或带初始文本的文本区域。

8.11. ComboBox (组合框)

ComboBox 允许用户从一个列表中选择个项目，也称为选择列表或下拉列表。

- **属性：** `value` (选中的值)、 `editable` (是否可编辑)、 `onAction` (动作事件处理器)、 `items` (列表项)、 `visibleRowCount` (可见行数)。
- **构造函数：** 创建空组合框或带指定列表项的组合框。

8.12. ListView (列表视图)

ListView 类似于 ComboBox，但它允许用户选择单个或多个值。

- **属性：** `items` (列表项)、 `orientation` (方向，水平/垂直)、 `selectionModel` (选择模型)。
- **构造函数：** 创建空列表视图或带指定列表项的列表视图。

8.13. ScrollBar (滚动条)

ScrollBar 允许用户从一个范围中选择一个值。滚动条有两种样式：水平和垂直。

- **属性：** `blockIncrement` (块增量)、 `max` (最大值)、 `min` (最小值)、 `unitIncrement` (单位增量)、 `value` (当前值)、 `visibleAmount` (可见部分大小)、 `orientation` (方向)。

8.14. Slider (滑块)

Slider 类似于 ScrollBar，但具有更多属性，并可以以多种形式出现。

- **属性：** 与 ScrollBar 类似，并增加了 `majorTickUnit` (主刻度单位)、 `minorTickCount` (次刻度数量)、 `showTickLabels` (显示刻度标签)、 `showTickMarks` (显示刻度标记)。

8.15. Case Study: TicTacToe (案例研究：井字棋)

井字棋游戏示例，包含 `Cell` 类表示棋盘上的单元格，以及 `TicTacToe` 类管理游戏逻辑和 UI。

8.16. Media (媒体)

- `Media` 类用于获取媒体源。
- `MediaPlayer` 类用于播放和控制媒体（如 `autoPlay` , `currentCount` , `cycleCount` , `mute` , `volume` , `totalDuration` 等属性）。
- `MediaView` 类是 `Node` 的子类，用于显示视频。

9. Chapter 14: JavaFX Basics (JavaFX 基础)

9.1. Motivations (动机)

JavaFX 是一个用于开发 Java GUI 程序的新框架。它展示了面向对象原则的优秀应用。本章旨在介绍 JavaFX 编程基础，并演示如何用 JavaFX 实现 OOP。具体而言，本章将介绍 JavaFX 框架以及 GUI 组件及其关系。

9.2. Objectives (目标)

本章目标包括：区分 JavaFX、Swing 和 AWT，编写简单的 JavaFX 程序并理解舞台 (stage)、场景 (scene) 和节点 (node) 之间的关系，使用窗格 (panes)、UI 控件和形状 (shapes) 创建用户界面，使用绑定属性 (binding properties) 同步属性值，使用通用属性 `style` 和 `rotate` 进行节点操作，使用 `Color` 类创建颜色，使用 `Font` 类创建字体，使用 `Image` 和 `ImageView` 类创建和显示图像，使用 `Pane`、`StackPane`、`FlowPane`、`GridPane`、`BorderPane`、`HBox` 和 `VBox` 布局节点，使用 `Text` 类显示文本并使用 `Line`、`Circle`、`Rectangle`、`Ellipse`、`Arc`、`Polygon` 和 `Polyline` 创建形状，以及开发可重用 GUI 组件 `ClockPane` 以显示模拟时钟。

9.3. JavaFX vs Swing and AWT (JavaFX 与 Swing 和 AWT 对比)

JavaFX 旨在取代 Swing 和 AWT，用于开发丰富的互联网应用程序。

- **AWT (Abstract Windows Toolkit)**：Java 早期 GUI 库，适用于简单 GUI，但易出现平台特定 bug。
- **Swing**：取代 AWT 的更健壮、多功能和灵活的库，组件直接绘制在画布上，对平台依赖性较低。
- **JavaFX**：Java 8 引入的全新 GUI 平台。

9.4. 创建 JavaFX 项目 (Creating JavaFX Projects)

可以通过以下方式创建 JavaFX 项目：

- Maven：使用 Maven archetype。
- VSCode 插件：
 - i. Extension Pack for Java (microsoft)
 - ii. Maven for Java
 - iii. Project Manager for Java
 - iv. Language Support for Java(TM) by Red Hat

9.5. Basic Structure of JavaFX (JavaFX 基本结构)

JavaFX 应用程序的基本结构包括：

- **Application (应用程序)**：JavaFX 程序的入口点。
- **Override the start(Stage) method (重写 start(Stage) 方法)**：应用程序的主入口点。
- **Stage (舞台)**：顶级容器，代表窗口。
- **Scene (场景)**：舞台中的内容容器，包含所有 UI 节点。
- **Nodes (节点)**：场景中的图形元素（如按钮、文本等）。

9.6. Panes, UI Controls, and Shapes (窗格、UI 控件和形状)

- **节点 (Node)**：所有可见元素（包括 UI 控件、形状等）的基类。
- **父节点 (Parent)**：可以包含其他节点的节点，如 Pane 和 Control。
- **窗格 (Pane)**：用于布局节点的容器，包括
FlowPane、GridPane、BorderPane、HBox、VBox、StackPane。
- **UI 控件 (Control)**：用户界面交互元素，如
Label、TextField、Button、CheckBox、RadioButton、TextArea。
- **形状 (Shape)**：图形元素，如
Line、Circle、Ellipse、Rectangle、Path、Polygon、Polyline、Text。
- **ImageView**：用于显示图像。

9.7. Display a Shape (显示形状)

JavaFX 使用传统的数学坐标系，y 轴向下。

9.8. Binding Properties (绑定属性)

JavaFX 引入了**绑定属性**的概念。它允许一个目标对象（或其属性）与一个源对象（或其属性）绑定。当源对象的值发生变化时，目标属性会自动更新。目标对象被称为**绑定对象**或**绑定属性**。

9.9. Uni/Bidirectional Binding (单向/双向绑定)

- **单向绑定**：源属性的变化影响目标属性，但目标属性的变化不影响源属性。
- **双向绑定**：源属性和目标属性相互影响，任何一方的变化都会同步到另一方。

9.10. Common Properties and Methods for Nodes (节点的通用属性和方法)

- **style**：用于设置 JavaFX CSS 样式。
- **rotate**：用于旋转节点。

9.11. The Color Class (Color 类)

`javafx.scene.paint.Color` 类用于表示颜色。

- **属性**：`red`、`green`、`blue` (0.0-1.0 范围) 和 `opacity` (透明度, 0.0-1.0 范围)。
- **构造函数**：根据 RGB 和透明度值创建颜色。
- **方法**：`brighter()` (更亮)、`darker()` (更暗)，以及获取不透明或 0-255 RGB 值的颜色。

9.12. The Font Class (Font 类)

`javafx.scene.text.Font` 类用于管理字体。

- **属性**：`size`、`name` 和 `family`。
- **构造函数**：可指定字体名称、大小、粗细、姿态等。
- **方法**：`getFamilies()` (获取字体家族列表)、`getFontNames()` (获取完整字体名称列表)。

9.13. The Image Class (Image 类)

`javafx.scene.image.Image` 类用于加载图像。

- **属性**：`error` (加载是否正确)、`height`、`width` (图像高度和宽度)、`progress` (加载进度)。
- **构造函数**：通过文件名或 URL 创建 `Image` 对象。

9.14. The ImageView Class (ImageView 类)

`javafx.scene.image.ImageView` 类是 `Node` 的子类，用于显示 `Image` 对象。

- **属性**：`fitHeight`、`fitWidth` (适应高度和宽度)、`x`、`y` (图像视图原点坐标)、`image` (要显示的图像)。

- **构造函数**：创建空的 `ImageView`、或带指定图像的 `ImageView`、或从文件/URL 加载图像的 `ImageView`。

9.15. Layout Panes (布局窗格)

JavaFX 提供多种窗格来组织容器中的节点。

- **Pane**：布局窗格的基类，包含 `getChildren()` 方法。
- **StackPane**：节点堆叠在彼此上方，居中显示。
- **FlowPane**：节点按行或列垂直/水平排列。
- **GridPane**：节点放置在二维网格的单元格中。
- **BorderPane**：节点放置在顶部、右侧、底部、左侧和中心区域。
- **HBox**：节点在一行中放置。
- **VBox**：节点在一列中放置。

9.16. FlowPane (流式窗格)

`FlowPane` 用于按行或列布局节点。

- **对齐方式 (alignment)**：默认 `Pos.LEFT`。
- **方向 (orientation)**：默认 `Orientation.HORIZONTAL`。
- **水平间距 (hgap)** 和 **垂直间距 (vgap)**。

9.17. GridPane (网格窗格)

`GridPane` 用于将节点放置在二维网格的单元格中。

- **对齐方式 (alignment)**：默认 `Pos.LEFT`。
- **网格线可见性 (gridLinesVisible)**：默认 `false`。
- **水平间距 (hgap)** 和 **垂直间距 (vgap)**。
- **添加节点 (add)**：指定子节点、列索引和行索引。

9.18. BorderPane (边框窗格)

`BorderPane` 用于将节点放置在顶部、右侧、底部、左侧和中心区域。

- 每个区域只能放置一个节点。
- 提供了设置每个区域节点和对齐方式的方法。

9.19. HBox (水平箱式窗格)

HBox 用于将节点排列在单行中。

- **对齐方式 (alignment)**: 默认 Pos.TOP_LEFT 。
- **填充高度 (fillHeight)**: 默认 true ，可调整子节点填充高度。
- **间距 (spacing)**: 子节点之间的水平间距。

9.20. VBox (垂直箱式窗格)

VBox 用于将节点排列在单列中。

- **对齐方式 (alignment)**: 默认 Pos.TOP_LEFT 。
- **填充宽度 (fillWidth)**: 默认 true ，可调整子节点填充宽度。
- **间距 (spacing)**: 子节点之间的垂直间距。

9.21. Shapes (形状)

JavaFX 提供多种形状类，用于绘制文本、线条、圆形、矩形、椭圆、弧、多边形和多段线。这些形状类都继承自 Node 。

9.22. Text (文本)

javafx.scene.text.Text 类用于显示文本。

- **属性**: text (要显示的文本)、 x 、 y (文本坐标)、 underline (下划线)、 strikethrough (删除线)、 font (字体)。
- **构造函数**: 创建空文本、或带指定文本的文本、或带指定坐标和文本的文本。

9.23. Line (线条)

javafx.scene.shape.Line 类用于绘制线条。

- **属性**: startX 、 startY (起点坐标)、 endX 、 endY (终点坐标)。
- **构造函数**: 创建空线条或带指定起终点坐标的线条。

9.24. Rectangle (矩形)

javafx.scene.shape.Rectangle 类用于绘制矩形。

- **属性**: x 、 y (左上角坐标)、 width 、 height (宽度和高度)、 arcWidth 、 arcHeight (圆弧宽度和高度，用于圆角矩形)。

- **构造函数**：创建空矩形或带指定属性的矩形。

9.25. Circle (圆形)

`javafx.scene.shape.Circle` 类用于绘制圆形。

- **属性**： `centerX`、 `centerY` (圆心坐标)、 `radius` (半径)。
- **构造函数**：创建空圆形或带指定圆心和半径的圆形。

9.26. Ellipse (椭圆)

`javafx.scene.shape.Ellipse` 类用于绘制椭圆。

- **属性**： `centerX`、 `centerY` (圆心坐标)、 `radiusX`、 `radiusY` (水平和垂直半径)。
- **构造函数**：创建空椭圆或带指定属性的椭圆。

9.27. Arc (弧)

`javafx.scene.shape.Arc` 类用于绘制弧。

- **属性**： `centerX`、 `centerY` (圆心坐标)、 `radiusX`、 `radiusY` (水平和垂直半径)、 `startAngle` (起始角度)、 `length` (弧长)、 `type` (闭合类型，如 `OPEN`，`CHORD`，`ROUND`)。
- **构造函数**：创建空弧或带指定属性的弧。

9.28. Polygon and Polyline (多边形和多段线)

- **Polygon (多边形)**：由一系列连接点组成的闭合形状。
- **Polyline (多段线)**：由一系列连接点组成的非闭合形状。
- 两者都通过 `getPoints()` 返回一个 `ObservableList<Double>`，存储点的 `x` 和 `y` 坐标。

9.29. Case Study: The ClockPane Class (案例研究：时钟窗格类)

`ClockPane` 类是一个自定义窗格，用于显示模拟时钟。

- **属性**： `hour`、 `minute`、 `second` (时、分、秒)。
- **方法**： `ClockPane()` (构造函数)、 `setCurrentTime()` (设置当前时间)、 `setWidth()`、 `setHeightTime()` (设置宽度和高度并重绘时钟)。

10. Chapter 15: Event-Driven Programming and Animations (事件驱动编程和动画)

10.1. Motivations (动机)

开发 GUI 程序需要处理用户交互。**事件驱动编程**是一种响应用户操作（如点击按钮）的编程范式。

10.2. Objectives (目标)

本章目标包括：理解事件驱动编程，描述事件、事件源和事件类，定义处理类、注册处理对象和编写事件处理代码，使用内部类和匿名内部类定义处理类，使用 Lambda 表达式简化事件处理，开发贷款计算器 GUI，处理鼠标事件 (MouseEvent) 和键盘事件 (KeyEvent)，创建监听器处理可观察对象的值变化，使用 Animation、PathTransition、FadeTransition 和 Timeline 类开发动画，以及模拟弹跳球动画。

10.3. Procedural vs. Event-Driven Programming (过程式 vs. 事件驱动编程)

- **过程式编程**：代码按程序顺序执行。
- **事件驱动编程**：代码在事件激活时执行。

10.4. Taste of Event-Driven Programming (事件驱动编程初体验)

一个简单的 GUI 示例，显示一个按钮。当按钮被点击时，控制台会显示消息，演示事件响应。

10.5. Handling GUI Events (处理 GUI 事件)

- **事件源对象 (Source object)**：生成事件的对象（例如，按钮）。
- **监听器对象 (Listener object)**：包含处理事件的方法。
- **事件**：由事件源对象触发。
- **处理程序 (Handler)**：监听器对象中的方法，实际处理事件。

10.6. Trace Execution (追踪执行)

一个 JavaFX 事件处理的追踪示例：

1. 从 main 方法开始，创建 GUI 窗口。
2. 点击“OK”按钮。
3. JVM 调用监听器的 handle 方法。

4. `handle` 方法中的代码执行，例如在控制台打印消息。

10.7. Events (事件)

- 事件可以定义为程序收到的一种信号，表明发生了某事。
- 事件通常由外部用户操作生成，例如鼠标移动、鼠标点击或键盘按键。

10.8. Event Classes (事件类)

JavaFX 事件类位于 `javafx.event` 包中。

事件类层次结构：

- `EventObject` (根) -> `Event`
 - `ActionEvent` (按钮点击等)
 - `InputEvent` (输入事件)
 - `MouseEvent` (鼠标事件)
 - `KeyEvent` (键盘事件)
 - `WindowEvent` (窗口事件)

10.9. Event Information (事件信息)

- 事件对象包含与事件相关的所有属性。
- 可以使用 `EventObject` 类中的 `getSource()` 方法识别事件源对象。
- `EventObject` 的子类处理特殊类型的事件，如按钮动作、窗口事件、鼠标移动和按键。

10.10. Selected User Actions and Handlers (选定的用户操作和处理程序)

表格列出了常见用户操作、事件源对象、事件类型以及相应的事件注册方法。

例如，按钮点击触发 `ActionEvent`，注册方法为 `setOnAction(EventHandler<ActionEvent>)`。鼠标事件和键盘事件则有更细分的类型和注册方法。

10.11. The Delegation Model (委托模型)

- **事件源**：生成事件并注册监听器。
- **监听器**：实现事件处理接口，包含处理事件的 `handle` 方法。
- **注册**：事件源将监听器对象注册到自身，以便在事件发生时调用 `handle` 方法。

10.12. The Delegation Model: Example (委托模型：示例)

示例代码展示了如何创建按钮 (Button)、处理类 (OKHandlerClass) 和事件注册 (setOnAction)。

```
Button btOK = new Button("OK"); //
OKHandlerClass handler = new OKHandlerClass(); //
btOK.setOnAction(handler); // 注册事件处理
```

10.13. Example: First Version for ControlCircle (no listeners) (示例：ControlCircle 的第一个版本（无监听器）)

演示一个包含两个按钮（放大、缩小）控制圆形大小的程序，但尚未添加监听器，因此按钮无效。

10.14. Example: Second Version for ControlCircle (with listener for Enlarge) (示例：ControlCircle 的第二个版本（带放大监听器）)

在第一个版本的基础上，为“放大”按钮添加了监听器，使其能够响应点击事件并改变圆形大小。

10.15. Inner Class Listeners (内部类监听器)

- 监听器类通常只用于特定 GUI 组件，不共享给其他应用程序。
- 适合将监听器类定义为框架类内部的**内部类 (inner class)**。

10.16. Inner Classes (内部类)

- **定义：**一个类是另一个类的成员。
- **优势：**
 - 使程序更简单。
 - 内部类可以访问其包含的外部类的数据和方法，无需传递外部类引用给内部类构造函数。

10.17. Inner Classes, cont. (内部类，续)

内部类使得程序更简洁。

内部类被编译成 OuterClassName\$InnerClassName.class 的形式。

内部类可以声明为 public、protected 或 private，遵循与成员相同的可见性规则。

静态内部类可以不依赖外部类实例而访问，但不能访问外部类的非静态成员。

10.18. Anonymous Inner Classes (匿名内部类)

- **定义：**没有名称的内部类，通常直接在创建实例时声明。
- **特性：**
 - 必须扩展一个超类或实现一个接口，但不能有显式的 `extends` 或 `implements` 子句。
 - 必须实现超类或接口中的所有抽象方法。
 - 总是使用其超类的无参构造函数创建实例（如果实现接口，则使用 `Object()` 的无参构造函数）。
 - 编译后名为 `OuterClassName$n.class`，其中 `n` 是数字。

10.19. Simplifying Event Handling Using Lambda Expressions (使用 Lambda 表达式简化事件处理)

Lambda 表达式是 Java 8 的新特性，可将匿名方法简化为更简洁的语法。它们是**单抽象方法接口 (SAM)** 的实现，即接口中只有一个抽象方法。

10.20. Basic Syntax for a Lambda Expression (Lambda 表达式基本语法)

- `(type1 param1, type2 param2, ...) -> expression`
- `(type1 param1, type2 param2, ...) -> { statements; }`
- 参数的数据类型可以显式声明或隐式推断。如果只有一个参数且数据类型可推断，则括号可以省略。

10.21. The MouseEvent Class (MouseEvent 类)

`javafx.scene.input.MouseEvent` 类用于处理鼠标事件。

- **信息：**按钮（左/中/右）、点击次数、`x/y` 坐标（事件源/场景/屏幕）、是否按下了 `Alt/Control/Meta/Shift` 键。

10.22. The KeyEvent Class (KeyEvent 类)

`javafx.scene.input.KeyEvent` 类用于处理键盘事件。

- **信息：**字符（`getCharacter()`）、键码（`getCode()`）、文本（`getText()`）、是否按下了 `Alt/Control/Meta/Shift` 键。

10.23. The KeyCode Constants (KeyCode 常量)

KeyCode 类定义了键盘按键的常量，例如 HOME , END , PAGE_UP , ENTER , F1 到 F12 , 0 到 9 , A 到 z 等。

10.24. Listeners for Observable Objects (可观察对象的监听器)

- 可以为**可观察对象 (Observable object)** 添加监听器，以处理其值的变化。
- 可观察对象具有 `addListener(InvalidationListener listener)` 方法。
- 监听器类应实现 `InvalidationListener` 接口，并使用 `invalidated(Observable o)` 方法处理属性值变化。

10.25. Animation (动画)

JavaFX 提供了 `Animation` 类作为所有动画的核心功能。

- **属性：** `autoReverse` (是否反向播放)、`cycleCount` (循环次数)、`rate` (播放速度)、`status` (动画状态)。
- **方法：** `pause()` (暂停)、`play()` (播放)、`stop()` (停止并重置)。

10.26. PathTransition (路径过渡)

`PathTransition` 类动画化一个节点沿着给定路径移动。

- **属性：** `duration` (持续时间)、`node` (目标节点)、`orientation` (节点方向)、`path` (形状路径)。
- **构造函数：** 可指定持续时间、路径和节点。

10.27. FadeTransition (淡入淡出过渡)

`FadeTransition` 类动画化节点透明度的变化。

- **属性：** `duration` (持续时间)、`node` (目标节点)、`fromValue` (起始透明度)、`toValue` (结束透明度)、`byValue` (透明度增量)。
- **构造函数：** 可指定持续时间、节点和透明度值。

10.28. Timeline (时间轴)

`Timeline` 类用于程序化地创建动画，通过一个或多个 `KeyFrame` 来实现。每个 `KeyFrame` 在指定的时间间隔内顺序执行。`Timeline` 继承自 `Animation`。

10.29. Case Study: Bouncing Ball (案例研究：弹跳球)

通过 `BallPane` 类表示一个弹跳球，并通过 `BounceBallControl` 类控制其运动，可以使用滑块控制速度。

11. Chapter 16: JavaFX UI Controls and Multimedia (JavaFX UI 控件和多媒体)

11.1. Motivations (动机)

图形用户界面 (GUI) 使系统用户友好且易于使用。创建 GUI 需要创造力和 GUI 组件的工作原理知识。Java 中的 GUI 组件非常灵活和多功能，可以创建各种有用的用户界面。

11.2. Objectives (目标)

本章目标包括：使用各种 UI 控件创建 GUI，使用 `Label` 及其属性，使用 `Button` 及其 `setOnAction`，使用 `CheckBox` 和 `RadioButton`（配合 `ToggleGroup`），使用 `TextField` 和 `PasswordField` 输入数据，使用 `TextArea` 输入多行数据，使用 `ComboBox` 单选，使用 `ListView` 单选/多选，使用 `ScrollBar` 和 `Slider` 选择范围，开发 Tic-Tac-Toe 游戏，使用 `Media`、`MediaPlayer` 和 `MediaView` 播放视频和音频，以及开发显示国旗和播放国歌的案例研究。

11.3. Frequently Used UI Controls (常用 UI 控件)

JavaFX 中常用的 UI 控件包括：`Label`，`Button`，`CheckBox`，`RadioButton`，`TextField`，`PasswordField`，`TextArea`，`ComboBox`，`ListView`，`ScrollBar`，`Slider`。这些控件通常继承自 `Control` 或 `Labeled`。

11.4. Labeled (带标签的控件)

`Labeled` 类是显示短文本、节点或两者皆有的显示区域的基类。`Label` 和 `Button` 等控件共享 `Labeled` 中定义的许多通用属性。

- **属性：** `alignment` (对齐方式)、`contentDisplay` (内容显示方式)、`graphic` (图形)、`graphicTextGap` (图形与文本间距)、`textFill` (文本颜色)、`text` (文本内容)、`underline` (下划线)、`wrapText` (文本换行)。

11.5. Label (标签)

Label 类定义了标签。

- **构造函数**：创建空标签、带文本的标签、或带文本和图形的标签。

11.6. ButtonBase and Button (按钮基类和按钮)

- **ButtonBase**：是触发动作事件的控件的基类。
- **Button**：触发点击事件的按钮。
- **共同特性**：onAction (动作事件处理器)。
- **构造函数**：创建空按钮，或带文本的按钮，或带文本和图形的按钮。

11.7. CheckBox (复选框)

CheckBox 用于用户进行选择。它继承了 ButtonBase 和 Labeled 的所有属性（如 onAction、text、graphic、alignment 等）。

- **属性**：selected (是否选中)。
- **构造函数**：创建空复选框或带文本的复选框。

11.8. RadioButton (单选按钮)

RadioButton 允许用户从一组选项中选择单个项目。它继承了 ToggleButton 的所有属性。

- **外观**：通常显示为一个圆形，选中时填充。
- **属性**：selected (是否选中)、toggleGroup (所属的单选组)。

11.9. TextField (文本字段)

TextField 用于输入或显示单行字符串。它是 TextInputControl 的子类。

- **属性**：text (文本内容)、editable (是否可编辑)、alignment (文本对齐)、prefColumnCount (首选列数)、onAction (动作事件处理器)。
- **构造函数**：创建空文本字段或带初始文本的文本字段。

11.10. TextArea (文本区域)

TextArea 允许用户输入多行文本。

- **属性**：prefColumnCount (首选列数)、prefRowCount (首选行数)、wrapText (是否换行)。
- **构造函数**：创建空文本区域或带初始文本的文本区域。

11.11. ComboBox (组合框)

ComboBox 允许用户从一个列表中选择一项，也称为选择列表或下拉列表。

- **属性：** value (选中的值)、 editable (是否可编辑)、 onAction (动作事件处理器)、 items (列表项)、 visibleRowCount (可见行数)。
- **构造函数：** 创建空组合框或带指定列表项的组合框。

11.12. ListView (列表视图)

ListView 类似于 ComboBox，但它允许用户选择单个或多个值。

- **属性：** items (列表项)、 orientation (方向，水平/垂直)、 selectionModel (选择模型)。
- **构造函数：** 创建空列表视图或带指定列表项的列表视图。

11.13. ScrollBar (滚动条)

ScrollBar 允许用户从一个范围中选择一个值。滚动条有两种样式：水平和垂直。

- **属性：** blockIncrement (块增量)、 max (最大值)、 min (最小值)、 unitIncrement (单位增量)、 value (当前值)、 visibleAmount (可见部分大小)、 orientation (方向)。

11.14. Slider (滑块)

Slider 类似于 ScrollBar，但具有更多属性，并可以以多种形式出现。

- **属性：** 与 ScrollBar 类似，并增加了 majorTickUnit (主刻度单位)、 minorTickCount (次刻度数量)、 showTickLabels (显示刻度标签)、 showTickMarks (显示刻度标记)。

11.15. Case Study: TicTacToe (案例研究：井字棋)

井字棋游戏示例，包含 Cell 类表示棋盘上的单元格，以及 TicTacToe 类管理游戏逻辑和 UI。

11.16. Media (媒体)

- Media 类用于获取媒体源。
- MediaPlayer 类用于播放和控制媒体 (如 autoPlay, currentCount, cycleCount, mute, volume, totalDuration 等属性)。
- MediaView 类是 Node 的子类，用于显示视频。

12. Chapter 17: Binary I/O (二进制 I/O)

12.1. Motivations (动机)

数据存储在文本文件中是人类可读的形式。数据存储在二进制文件中是二进制形式。你不能读取二进制文件。它们被设计为由程序读取。例如，Java 源代码存储在文本文件中，可以通过文本编辑器读取。而 Java 类存储在二进制文件中，由 JVM 读取。二进制文件的优势在于它们处理效率比文本文件更高。

12.2. Objectives (目标)

本章目标包括：了解 Java 中 I/O 处理方式，区分文本 I/O 和二进制 I/O，使用 `FileInputStream` 和 `FileOutputStream` 读写字节，使用 `DataInputStream` 和 `DataOutputStream` 读写基本类型值和字符串，使用 `ObjectOutputStream` 和 `ObjectInputStream` 存储和恢复对象（序列化），实现 `Serializable` 接口使对象可序列化，序列化数组，以及使用 `RandomAccessFile` 读写随机位置的文件。

12.3. How is I/O Handled in Java? (Java 中如何处理 I/O?)

`File` 对象封装文件或路径的属性，但不包含读写数据的方法。为了执行 I/O 操作，你需要使用适当的 Java I/O 类创建对象。这些对象包含读写文件数据的方法。本节介绍如何使用 `Scanner` 和 `PrintWriter` 类读写文本文件中的字符串和数值。

12.4. Text File vs. Binary File (文本文件 vs. 二进制文件)

数据存储 in 文本文件中是人类可读的形式。数据存储在二进制文件中是二进制形式。你不能读取二进制文件。二进制文件的优势在于它们处理效率比文本文件更高。

虽然在技术上不精确但正确地说，你可以想象文本文件由字符序列组成，二进制文件由位序列组成。例如，十进制整数 199 在文本文件中存储为三个字符 '1'、'9'、'9' 的序列，而在二进制文件中存储为字节类型值 C7，因为十进制 199 等于十六进制 C7。

12.5. Binary I/O (二进制 I/O)

文本 I/O 需要编码和解码。JVM 在写入字符时将 Unicode 转换为文件特定编码，在读取字符时将文件特定编码转换为 Unicode。二进制 I/O 不需要转换。当你向文件写入一个字节时，原始字节被复制到文件中。当你从文件读取一个字节时，返回精确的字节。

12.6. Binary I/O Classes (二进制 I/O 类)

Java I/O 流的层次结构：

- Object -> Throwable
 - InputStream (输入流基类)
 - FileInputStream (文件输入流)
 - FilterInputStream (过滤输入流)
 - DataInputStream (数据输入流)
 - BufferedInputStream (缓冲输入流)
 - ObjectInputStream (对象输入流)
 - OutputStream (输出流基类)
 - FileOutputStream (文件输出流)
 - FilterOutputStream (过滤输出流)
 - DataOutputStream (数据输出流)
 - BufferedOutputStream (缓冲输出流)
 - ObjectOutputStream (对象输出流)
 - PrintStream (打印流)

12.7. InputStream (输入流)

`java.io.InputStream` 是所有输入流的抽象基类。

- `read()`：读取下一个字节，返回 `int` 类型 (0-255)，如果到达文件末尾返回 -1。
- `read(b: byte[])`：读取字节到字节数组 `b`。
- `read(b: byte[], off: int, len: int)`：读取指定长度的字节到字节数组的偏移量处。
- 其他方法包括 `available()` (可用字节数)、`close()` (关闭流)、`skip()` (跳过字节)、`markSupported()` (是否支持标记)、`mark()` (标记当前位置)、`reset()` (重置到标记位置)。

12.8. OutputStream (输出流)

`java.io.OutputStream` 是所有输出流的抽象基类。

- `write(int b)`：将指定字节写入输出流。
- `write(b: byte[])`：将字节数组中的所有字节写入输出流。
- `write(b: byte[], off: int, len: int)`：将字节数组中指定范围的字节写入输出流。
- 其他方法包括 `close()` (关闭流)、`flush()` (刷新缓冲区)。

12.9. FileInputStream/FileOutputStream (文件输入/输出流)

- `FileInputStream` 和 `FileOutputStream` 将二进制输入/输出流与外部文件关联。
- 它们的方法都继承自其超类 `InputStream` 和 `OutputStream`。
- `FileInputStream` 构造函数：`public FileInputStream(String filename)` 或 `public FileInputStream(File file)`。

- **FileNotFoundException**：尝试创建不存在文件的 `FileInputStream` 时会发生此异常。
- **FileOutputStream 构造函数**：可以指定文件名/文件对象，以及是否以追加模式打开文件（`boolean append`）。
 - 如果文件不存在，则创建新文件。
 - 如果文件存在，默认情况下会覆盖其内容；若要追加内容，`append` 参数需设为 `true`。

12.10. FilterInputStream/FilterOutputStream (过滤输入/输出流)

- **过滤流**：用于过滤字节流。
- **目的**：将原始字节流包装起来，以提供更高级的读写功能（如读写整数、双精度浮点数或字符串）。
- **FilterInputStream** 和 **FilterOutputStream** 是过滤数据的基础类。
- **DataInputStream** 和 **DataOutputStream** 用于处理基本数值类型。

12.11. DataInputStream/DataOutputStream (数据输入/输出流)

- **DataInputStream**：从流中读取字节并将其转换为相应的基本类型值或字符串。
- **DataOutputStream**：将基本类型值或字符串转换为字节并输出到流中。

12.12. DataInputStream (数据输入流)

`DataInputStream` 扩展 `FilterInputStream` 并实现 `DataInput` 接口。

提供了读取各种基本数据类型（如 `boolean`，`byte`，`char`，`float`，`double`，`int`，`long`，`short`）和字符串（`readLine()`，`readUTF()`）的方法。

12.13. DataOutputStream (数据输出流)

`DataOutputStream` 扩展 `FilterOutputStream` 并实现 `DataOutput` 接口。

提供了写入各种基本数据类型和字符串（`writeUTF()`）的方法。

12.14. Characters and Strings in Binary I/O (二进制 I/O 中的字符和字符串)

- `writeChar(char c)` 写入 Unicode 字符（两字节）。
- `writeChars(String s)` 写入字符串中每个字符的 Unicode 形式。
- **UTF-8**：一种编码方案，可高效处理 ASCII 和 Unicode。
 - ASCII 值（`<0x7F`）编码为一字节。
 - Unicode 值（`<0x7FF`）编码为两字节。
 - 其他 Unicode 值编码为三字节。

12.15. Using DataInputStream/DataOutputStream (使用数据输入/输出流)

数据流作为现有输入/输出流的包装器，用于过滤数据。

- **构造函数：** `DataInputStream(InputStream instream)` 和 `DataOutputStream(OutputStream outstream)` 。

12.16. Concept of pipe line (管道概念)

I/O 操作可以看作是数据通过一系列流进行处理的“管道”：

`DataInputStream` -> `FileInputStream` -> External File (读取)。

`DataOutputStream` -> `FileOutputStream` -> External File (写入)。

12.17. Order and Format (顺序和格式)

注意：必须以写入时的相同顺序和格式读取数据。

例如，如果使用 `writeUTF` 写入字符串，则必须使用 `readUTF` 读取。

12.18. Checking End of File (检查文件末尾)

提示：如果持续读取数据直到流末尾，可能会发生 `EOFException` 。

可以使用 `input.available() == 0` 来检查是否到达文件末尾。

12.19. BufferedInputStream/BufferedOutputStream (缓冲输入/输出流)

`BufferedInputStream` 和 `BufferedOutputStream` 不包含新方法。它们的所有方法都继承自 `InputStream` 和 `OutputStream` 类。缓冲流通过使用缓冲区来加速 I/O 操作。

12.20. Constructing BufferedInputStream/BufferedOutputStream (构造缓冲输入/输出流)

- **BufferedInputStream 构造函数：** `public BufferedInputStream(InputStream in)` 或 `public BufferedInputStream(InputStream in, int bufferSize)` 。
- **BufferedOutputStream 构造函数：** `public BufferedOutputStream(OutputStream out)` 或 `public BufferedOutputStream(OutputStream out, int bufferSize)` 。

12.21. Case Studies: Copy File (案例研究：复制文件)

本案例研究开发了一个复制文件的程序。用户需要提供源文件和目标文件作为命令行参数。程序将复制源文件到目标文件，并显示文件中的字节数。如果源文件不存在，将提示用户文件未找到。如果目标文件已存在，将提示用户目标文件已存在。

12.22. Object I/O (对象 I/O)

`DataInputStream / DataOutputStream` 能够进行基本类型值和字符串的 I/O 操作。

`ObjectInputStream / ObjectOutputStream` 除了基本类型值和字符串，还能进行对象的 I/O 操作。

12.23. ObjectInputStream (对象输入流)

`ObjectInputStream` 扩展 `InputStream` 并实现 `ObjectInput` 和 `ObjectStreamConstants`。

- `readObject()` : 读取一个对象。

12.24. ObjectOutputStream (对象输出流)

`ObjectOutputStream` 扩展 `OutputStream` 并实现 `ObjectOutput` 和 `ObjectStreamConstants`。

- `writeObject(o: Object)` : 写入一个对象。

12.25. Using Object Streams (使用对象流)

你可以将 `ObjectInputStream / ObjectOutputStream` 包装在任何 `InputStream / OutputStream` 上。

- `ObjectInputStream` 构造函数: `public ObjectInputStream(InputStream in)`。
- `ObjectOutputStream` 构造函数: `public ObjectOutputStream(OutputStream out)`。

12.26. The Serializable Interface (Serializable 接口)

并非所有对象都能写入输出流。能写入对象流的对象被称为**可序列化的**。一个可序列化对象是 `java.io.Serializable` 接口的实例。因此，一个可序列化对象的类必须实现 `Serializable` 接口。

`Serializable` 接口是一个**标记接口**。它没有方法，所以你不需要在实现 `Serializable` 的类中添加额外的代码。

实现此接口使 Java 序列化机制能够自动化存储对象和数组的过程。

12.27. The transient Keyword (transient 关键字)

如果一个对象是 `Serializable` 的实例，但它包含不可序列化的实例数据字段，那么该对象可以被序列化吗？答案是不能。为了使对象可序列化，你可以使用 `transient` 关键字标记这些数据字段，告诉 JVM 在将对象写入对象流时忽略这些字段。

12.28. The transient Keyword, cont. (transient 关键字，续)

考虑以下类：

```
public class Foo implements java.io.Serializable {  
    private int v1;  
    private static double v2;  
    private transient A v3 = new A();  
}  
class A { } // A is not serializable
```

当 `Foo` 类的对象被序列化时，只有变量 `v1` 被序列化。变量 `v2` 没有被序列化，因为它是一个静态变量。变量 `v3` 没有被序列化，因为它被标记为 `transient`。如果 `v3` 没有被标记为 `transient`，就会发生 `java.io.NotSerializableException`。

12.29. Serializing Arrays (序列化数组)

如果数组的所有元素都可序列化，则该数组是可序列化的。因此，整个数组可以使用 `writeObject` 写入文件，然后使用 `readObject` 恢复。

12.30. Random Access Files (随机访问文件)

你目前使用的所有流都是只读或只写流。外部文件是顺序文件，不能在不创建新文件的情况下更新。通常需要修改文件或向文件插入新记录。Java 提供了 `RandomAccessFile` 类，允许在随机位置读写文件。

12.31. RandomAccessFile (随机访问文件)

`java.io.RandomAccessFile` 类提供了以下功能：

- **构造函数：** `RandomAccessFile(file: File, mode: String)` 或 `RandomAccessFile(name: String, mode: String)`。
- `close()`：关闭流。
- `getFilePointer()`：返回文件指针的当前偏移量。
- `length()`：返回文件的长度。

- `read()` : 读取字节。
- `read(b: byte[])` : 读取字节到数组。
- `read(b: byte[], off: int, len: int)` : 读取指定长度的字节到数组。
- `seek(pos: long)` : 设置文件指针偏移量。
- `setLength(newLength: long)` : 设置文件长度。
- `skipBytes(int n)` : 跳过字节。
- `write(b: byte[])` : 写入字节数组。
- `write(b: byte[], off: int, len: int)` : 写入指定长度的字节数组。

12.32. File Pointer (文件指针)

随机访问文件由字节序列组成。有一个特殊的标记，称为**文件指针**，它定位在其中一个字节处。读或写操作发生在文件指针所在的位置。文件打开时，文件指针位于文件开头。当你读写数据时，文件指针会向前移动到下一个数据位置。例如，如果你使用 `readInt()` 读取一个 `int` 值，JVM 会从文件指针处读取四个字节，然后文件指针会比之前的位置向前移动四个字节。

12.33. RandomAccessFile Methods (随机访问文件方法)

`RandomAccessFile` 中的许多方法与 `DataInputStream` 和 `DataOutputStream` 中的方法相同。例如，`readInt()`，`readLong()`，`writeDouble()`，`readLine()`，`writeInt()`，和 `writeLong()` 都可以用于数据输入流或数据输出流以及 `RandomAccessFile` 流。

12.34. RandomAccessFile Methods, cont. (随机访问文件方法，续)

- `void seek(long pos) throws IOException`; 设置从文件开头开始的偏移量，下一个读写操作将在此处进行。
- `long getFilePointer() throws IOException`; 返回文件指针的当前偏移量（以字节为单位），从文件开头开始，下一个读写操作将在此处进行。
- `long length() throws IOException` 返回文件的长度。
- `final void writeChar(int v) throws IOException` 将一个字符（两字节 Unicode）写入文件，高字节在前。
- `final void writeChars(String s) throws IOException` 将一个字符串作为字符序列写入文件。

12.35. RandomAccessFile Constructor (RandomAccessFile 构造函数)

- `RandomAccessFile raf = new RandomAccessFile("test.dat", "rw");` // 允许读写。
- `RandomAccessFile raf = new RandomAccessFile("test.dat", "r");` // 只读。

13. Chapter 19: Generics (泛型)

13.1. Objectives (目标)

本章目标包括：了解泛型的好处，使用泛型类和接口，声明泛型类和接口，理解泛型类型如何提高可靠性和可读性，声明和使用泛型方法和有界泛型类型，使用原始类型以实现向后兼容性，了解通配符类型以及它们为何必要，使用 JDK 1.5 泛型转换旧版代码，理解泛型类型信息被编译器擦除以及泛型类的所有实例共享相同的运行时类文件，了解由类型擦除引起的泛型类型限制，以及设计和实现泛型矩阵类。

13.2. Why Do You Get a Warning? (为什么会有警告?)

当你使用未指定泛型类型的集合（即使用原始类型）时，编译器会发出警告。例如：

```
public class ShowUncheckedWarning {  
    public static void main(String[] args) {  
        java.util.ArrayList list = new java.util.ArrayList(); // 原始类型  
        list.add("Java Programming");  
    }  
}
```

为了理解这条线上的编译警告，你需要学习 JDK 1.6 泛型。

13.3. Fix the Warning (修复警告)

通过指定泛型类型，可以消除编译警告。

```
public class ShowUncheckedWarning {  
    public static void main(String[] args) {  
        java.util.ArrayList<String> list = new java.util.ArrayList<String>(); // 指定泛型类型  
        list.add("Java Programming");  
    }  
}
```

现在没有编译警告了。

13.4. What is Generics? (什么是泛型?)

泛型是参数化类型的能力。通过此能力，你可以定义一个带有泛型类型（可以由编译器替换为具体类型）的类或方法。例如，你可以定义一个存储泛型类型元素的泛型栈类。从这个泛型类中，你可以创建一个存储字符串的栈对象和一个存储数字的栈对象。在这里，字符串和数字是替代泛型类型的具体类型。

13.5. Why Generics? (为什么使用泛型?)

泛型的主要好处是能够在编译时而非运行时检测到错误。泛型类或方法允许你指定类或方法可以使用的允许对象类型。如果你尝试将类或方法与不兼容的对象一起使用，就会发生编译错误。

13.6. Generic Type (泛型类型)

泛型接口示例：

```
package java.lang;
public interface Comparable<T> { // 泛型接口
    public int compareTo (T o);
}
```

泛型实例化：在创建对象时指定具体类型。

13.7. Generic Instantiation (泛型实例化)

使用泛型类型可以提高可靠性并实现编译时错误检查。

- **Prior to JDK 1.5 (JDK 1.5 之前):**

```
Comparable c = new Date();
System.out.println(c.compareTo("red")); // 可能导致运行时错误
```

- **JDK 1.5 (JDK 1.5 之后):**

```
Comparable<Date> c = new Date();
System.out.println(c.compareTo("red")); // 编译错误 (因为 "red" 不是 Date 类型)
```

这提高了可靠性，并在编译时捕获错误。

13.8. Generic ArrayList in JDK 1.5 (JDK 1.5 中的泛型 ArrayList)

- **JDK 1.5 之前：** `java.util.ArrayList` 中的方法参数和返回值都是 `Object` 类型，需要手动类型转换。

- **JDK 1.5 之后：** `java.util.ArrayList<E>` 使用泛型，方法参数和返回值都是类型参数 `E`，无需手动类型转换。

13.9. No Casting Needed (无需类型转换)

使用泛型 `ArrayList` 时，无需进行显式类型转换，Java 会自动处理。

```
ArrayList<Double> list = new ArrayList<>();
list.add(5.5); // 5.5 自动转换为 new Double(5.5)
list.add(3.0); // 3.0 自动转换为 new Double(3.0)
Double doubleObject = list.get(0); // 无需类型转换
double d = list.get(1); // 自动转换为 double
```

13.10. Declaring Generic Classes and Interfaces (声明泛型类和接口)

可以通过在类名或接口名后加上 `<E>` (或其他类型参数名) 来声明泛型类和接口。

例如： `GenericStack<E>`

- **数据字段：** `list: java.util.ArrayList<E>`
- **方法：** `GenericStack()` (构造函数)、 `getSize(): int`、 `peek(): E`、 `pop(): E`、 `push(o: E): void`、 `isEmpty(): boolean`。

13.11. Generic Methods (泛型方法)

方法也可以声明为泛型，允许它们处理不同类型的参数。

```
public static <E> void print(E[] list) { // 泛型方法
    for (int i = 0; i < list.length; i++)
        System.out.print(list[i] + " ");
    System.out.println();
}
```

对比非泛型方法，泛型方法提供了更好的类型安全和代码重用。

13.12. Bounded Generic Type (有界泛型类型)

泛型类型参数可以被限定在一个特定的类型或其子类型中。

语法： `<E extends GeometricObject>`

这意味着 `E` 必须是 `GeometricObject` 或其子类。

```
public static <E extends GeometricObject> boolean // 有界泛型
    equalArea(E object1, E object2) {
    return object1.getArea() == object2.getArea();
}
```

13.13. Raw Type and Backward Compatibility (原始类型和向后兼容性)

- **原始类型 (Raw Type)**: 在泛型引入之前, 不指定类型参数的泛型类型被称为原始类型。
- 例如: `ArrayList list = new ArrayList();`
- 这大致等同于 `ArrayList<Object> list = new ArrayList<Object>();`
- 原始类型是为了向后兼容 JDK 1.5 之前的代码而存在的。

13.14. Raw Type is Unsafe (原始类型不安全)

使用原始类型可能导致运行时错误, 因为编译器无法在编译时进行严格的类型检查。

```
public class Max {
    public static Comparable max(Comparable o1, Comparable o2) { ... }
}
Max.max("Welcome", 23); // 可能导致运行时错误, 因为类型不兼容
```

运行时错误: `ClassCastException`, 因为 `23` 无法与 `String` 进行比较。

13.15. Make it Safe (使其安全)

使用有界泛型类型可以确保类型安全, 并在编译时捕获类型不兼容的错误。

```
public class Max1 {
    public static <E extends Comparable<E>> E max(E o1, E o2) { ... } // 有界泛型
}
Max1.max("Welcome", 23); // 编译错误, 因为 23 不是 Comparable<String>
```

13.16. Wildcards (通配符)

通配符 (`?`) 用于表示未知类型, 增加泛型代码的灵活性。

- `?`: **无界通配符** (unbounded wildcard), 表示任何类型。
- `? extends T`: **有界通配符** (bounded wildcard), 表示 `T` 类型或其子类型。

- `? super T` : **下限通配符** (lower bound wildcard), 表示 `T` 类型或其超类型。

13.17. Generic Types and Wildcard Types (泛型类型和通配符类型)

这些图示展示了不同通配符类型之间的关系和层次结构。

`Object` 是所有类型的根。

`A<?>` 可以表示 `A<String>`、`A<Integer>` 等。

`A<? extends B>` 可以表示 `A<B>`、`A<B's subclass>`。

`A<? super B>` 可以表示 `A<B>`、`A<B's superclass>`。

13.18. Avoiding Unsafe Raw Types (避免不安全的原始类型)

为了类型安全，应始终使用泛型类型参数创建集合，而不是原始类型。

- 使用： `new ArrayList<ConcreteType>()`
- 而不是： `new ArrayList();`

13.19. Erasure and Restrictions on Generics (类型擦除和泛型限制)

泛型通过**类型擦除 (type erasure)** 实现。编译器使用泛型类型信息来编译代码，但之后会擦除它。因此，泛型信息在运行时不可用。这种方法使泛型代码能够与使用原始类型的旧版代码向后兼容。

13.20. Compile Time Checking (编译时检查)

编译器在编译时检查泛型是否正确使用，并将其转换为运行时使用的等效代码（使用原始类型）。

```
// (a) 泛型代码
ArrayList<String> list = new ArrayList<>();
list.add("Oklahoma");
String state = list.get(0);
```

```
// (b) 编译后等效的原始类型代码
ArrayList list = new ArrayList();
list.add("Oklahoma");
String state = (String) (list.get(0));
```

编译器负责在编译时进行类型检查和类型转换的插入。

13.21. Important Facts (重要事实)

泛型类被其所有实例共享，无论其实际泛型类型如何。

例如，`GenericStack<String>` 和 `GenericStack<Integer>` 是两种类型，但 JVM 中只加载了一个 `GenericStack` 类。这是类型擦除的体现。

13.22. Restrictions on Generics (泛型限制)

1. 不能创建泛型类型的实例：例如 `new E()` 不允许。
2. 不允许创建泛型数组：例如 `new E[100]` 不允许。
3. 类的泛型类型参数不允许在静态上下文中使用。
4. 异常类不能是泛型。

13.23. Designing Generic Matrix Classes (设计泛型矩阵类)

目标：本示例提供一个泛型类，用于矩阵算术。该类实现了适用于所有类型矩阵的矩阵加法和乘法。

13.24. UML Diagram (UML 图)

`GenericMatrix<E extends Number>` 类的 UML 图，它是一个泛型类，用于矩阵操作。

- 数据字段：
`#add(element1: E, element2: E): E`、`#multiply(element1: E, element2: E): E`、`#zero(): E`。
- 方法：`+addMatrix(...)`、`+multiplyMatrix(...)`、`+printResult(...)`。
- `IntegerMatrix` 和 `RationalMatrix` 是其子类。

13.25. Source Code (源代码)

目标：本示例提供两个程序，利用 `GenericMatrix` 类进行整数矩阵算术和有理数矩阵算术。

14. Chapter 20: Lists, Stacks, Queues, and Priority Queues (列表、栈、队列和优先队列)

14.1. Objectives (目标)

本章目标包括：探索 Java Collections Framework 层次结构中接口和类之间的关系，使用 `Collection` 接口中定义的常用方法操作集合，使用 `Iterator` 接口遍历集合中的元素以及使用 `for-each` 循环遍历集

合，探讨如何以及何时使用 `ArrayList` 或 `LinkedList` 存储元素，使用 `Comparable` 接口和 `Comparator` 接口比较元素，使用 `Collections` 类中的静态实用方法进行排序、搜索、洗牌和查找集合中最大和最小元素，开发一个多重弹跳球应用程序使用 `ArrayList`，区分 `Vector` 和 `ArrayList` 以及使用 `Stack` 类创建栈，探索 `Collection`、`Queue`、`LinkedList` 和 `PriorityQueue` 之间的关系，并使用 `PriorityQueue` 类创建优先队列，以及使用栈编写程序来评估表达式。

14.2. What is Data Structure? (什么是数据结构?)

数据结构是按照某种方式组织的数据集合。该结构不仅存储数据，还支持访问和操作数据的功能。

14.3. Java Collection Framework hierarchy (Java 集合框架层次结构)

集合是存储一组对象的容器对象，通常称为元素。Java 集合框架支持三种类型的集合：列表 (lists)、集 (sets) 和映射 (maps)。

14.4. Java Collection Framework hierarchy, cont. (Java 集合框架层次结构，续)

`Set` 和 `List` 是 `Collection` 的子接口。

层次结构概览：

- `Collection` (接口)
 - `Set` (接口) -> `SortedSet` (接口) -> `NavigableSet` (接口) -> `TreeSet` (类)
 - `List` (接口) -> `AbstractList` (抽象类)
 - `ArrayList` (类)
 - `LinkedList` (类) -> `AbstractSequentialList` (抽象类) -> `Deque` (接口)
 - `Queue` (接口) -> `AbstractQueue` (抽象类)
 - `PriorityQueue` (类)
- 其他类：`AbstractCollection`、`AbstractSet`、`HashSet`、`LinkedHashSet`、`Vector`、`Stack`。

14.5. The Collection Interface (Collection 接口)

`Collection` 接口用于操作对象的集合。

- `iterator()`：返回一个迭代器，用于遍历集合中的元素。
- `forEach(action)`：对集合中的每个元素执行一个操作。
- `add(e)`：向集合添加元素。
- `addAll(c)`：将另一个集合的所有元素添加到当前集合。
- `clear()`：删除所有元素。

- `contains(o)` : 检查是否包含特定元素。
- `containsAll(c)` : 检查是否包含另一个集合的所有元素。
- `isEmpty()` : 检查集合是否为空。
- `remove(o)` : 删除特定元素。
- `removeAll(c)` : 删除另一个集合的所有元素。
- `retainAll(c)` : 只保留与另一个集合共有的元素。
- `size()` : 返回元素数量。
- `toArray()` : 返回包含所有元素的数组。
- `stream()` / `parallelStream()` : 返回流（在 Chapter 23 中介绍）。

14.6. The List Interface (List 接口)

`List` 存储元素按顺序排列，并允许用户指定元素存储位置。用户可以通过索引访问元素。

14.7. The List Interface, cont. (List 接口，续)

`List` 接口扩展了 `Collection` 接口，并增加了基于索引的操作。

- `add(index, element)` : 在指定索引处添加元素。
- `addAll(index, c)` : 在指定索引处添加另一个集合的所有元素。
- `get(index)` : 返回指定索引处的元素。
- `indexOf(o)` : 返回第一个匹配元素的索引。
- `lastIndexOf(o)` : 返回最后一个匹配元素的索引。
- `listIterator()` : 返回列表迭代器。
- `listIterator(startIndex)` : 返回从指定索引开始的列表迭代器。
- `remove(index)` : 删除指定索引处的元素。
- `set(index, element)` : 设置指定索引处的元素。
- `subList(fromIndex, toIndex)` : 返回子列表。

14.8. The List Iterator (列表迭代器)

`java.util.ListIterator<E>` 扩展了 `Iterator<E>`，提供了双向遍历列表和修改列表的功能。

- `add(element)` : 向列表添加元素。
- `hasPrevious()` : 检查是否有上一个元素。
- `nextIndex()` : 返回下一个元素的索引。
- `previous()` : 返回上一个元素。
- `previousIndex()` : 返回上一个元素的索引。
- `set(element)` : 替换上次 `next()` 或 `previous()` 返回的元素。

14.9. ArrayList and LinkedList (ArrayList 和 LinkedList)

ArrayList 和 LinkedList 是 List 接口的具体实现。

- **ArrayList** :
 - 基于数组实现，支持通过索引进行随机访问（get 和 set）效率高。
 - 插入或删除非末尾元素效率低。
- **LinkedList** :
 - 基于链表实现，支持在任意位置插入或删除元素效率高。
 - 随机访问（get 和 set）效率低。
- 列表可以动态增长或收缩。数组一旦创建大小就固定。

14.10. The Comparator Interface (Comparator 接口)

有时需要比较不同类型的元素，或者元素不是 Comparable 类型。

java.util.Comparator 接口定义了比较两个对象的方法。

- **compare(Object element1, Object element2)** : 返回负数、零或正数，表示 element1 小于、等于或大于 element2。

14.11. Other Comparator Examples (其他比较器示例)

可以创建自定义 Comparator 来定义不同的排序规则，例如按字符串长度排序或忽略大小写排序。

14.12. The Collections Class (Collections 类)

Collections 类包含各种静态方法，用于操作集合和映射、创建同步集合以及创建只读集合。

14.13. The Collections Class UML Diagram (Collections 类 UML 图)

java.util.Collections 类提供了大量用于操作或返回集合的静态方法，包括：

- **排序**：sort(list)、sort(list, comparator)。
- **搜索**：binarySearch(list, key)、binarySearch(list, key, comparator)。
- **反转**：reverse(list)。
- **洗牌**：shuffle(list)、shuffle(list, random)。
- **复制**：copy(dest, src)。
- **频率/最大/最小**：frequency(c, o)、max(c)、min(c)。
- **不相交**：disjoint(c1, c2)。

14.14. Case Study: Multiple Bouncing Balls (案例研究：多重弹跳球)

这个案例研究开发了一个多重弹跳球的应用程序，可能使用了 `ArrayList` 来管理多个球对象。

14.15. The Vector and Stack Classes (Vector 和 Stack 类)

Java 集合框架是在 Java 2 中引入的。在此之前，`Vector` 和 `Stack` 等数据结构已受支持。这些类经过重新设计以适应 Java 集合框架，但其旧式方法仍保留以保持兼容性。

14.16. The Vector Class (Vector 类)

在 Java 2 中，`Vector` 与 `ArrayList` 类似，但 `Vector` 包含用于访问和修改向量的**同步方法**。目前引入的新集合数据结构都不是同步的。如果需要同步，可以使用集合类的同步版本。

14.17. The Vector Class, cont. (Vector 类，续)

`java.util.Vector<E>` 类提供了：

- **构造函数**：创建空向量、从集合创建、或指定初始容量和容量增量。
- **添加元素**：`addElement(o)`。
- **容量/大小**：`capacity()`、`size()`、`trimToSize()`、`setSize()`。
- **访问元素**：`elementAt(index)`、`firstElement()`、`lastElement()`。
- **枚举**：`elements()`。
- **插入/删除**：
除：`insertElementAt()`、`removeAllElements()`、`removeElement()`、`removeElementAt()`、`setElementAt()`。

14.18. The Stack Class (Stack 类)

`Stack` 类表示一种****后进先出（LIFO）****的堆栈对象。元素只能从堆栈顶部访问。你可以从堆栈顶部检索、插入或删除元素。

- **`Stack()`**：创建一个空栈。
- **`empty()`**：检查栈是否为空。
- **`peek()`**：返回栈顶元素但不删除。
- **`pop()`**：返回并删除栈顶元素。
- **`push(o)`**：将元素添加到栈顶。
- **`search(o)`**：返回指定元素在栈中的位置。

14.19. Queues and Priority Queues (队列和优先队列)

队列 (Queue) 是一种先进先出 (FIFO) 的数据结构。元素在队列末尾追加，从队列开头删除。在优先队列 (Priority Queue) 中，元素被分配了优先级。访问元素时，优先级最高的元素首先被删除。

14.20. The Queue Interface (Queue 接口)

`java.util.Queue<E>` 接口扩展了 `Collection<E>`，并定义了队列操作：

- `offer(element)`：插入元素到队列。
- `poll()`：检索并删除队列头，如果队列为空则返回 `null`。
- `remove()`：检索并删除队列头，如果队列为空则抛出异常。
- `peek()`：检索队列头但不删除，如果队列为空则返回 `null`。
- `element()`：检索队列头但不删除，如果队列为空则抛出异常。

14.21. Using LinkedList for Queue (使用 LinkedList 作为队列)

`java.util.LinkedList<E>` 类实现了 `List<E>`、`Queue<E>` 和 `Deque<E>` 接口，因此可以作为队列使用。

14.22. The PriorityQueue Class (PriorityQueue 类)

`java.util.PriorityQueue<E>` 类实现了 `Queue<E>` 接口，提供优先队列功能。

- 构造函数：
 - `PriorityQueue()`：创建默认容量为 11 的优先队列。
 - `PriorityQueue(initialCapacity)`：创建指定初始容量的优先队列。
 - `PriorityQueue(c: Collection)`：从集合创建优先队列。
 - `PriorityQueue(initialCapacity, comparator)`：创建指定初始容量和比较器的优先队列。

14.23. Case Study: Evaluating Expressions (案例研究：表达式求值)

栈可用于评估表达式。

14.24. Algorithm (算法)

阶段 1：扫描表达式

程序从左到右扫描表达式以提取操作数、运算符和括号。

1.1. 如果提取的项目是操作数，将其推入 `operandStack`。

1.2. 如果提取的项目是 `+` 或 `-` 运算符，处理 `operatorStack` 顶部所有操作符，然后将提取的操作符

推入 `operatorStack` 。

1.3. 如果提取的项目是 `*` 或 `/` 运算符，处理 `operatorStack` 顶部所有 `*` 或 `/` 操作符，然后将提取的操作符推入 `operatorStack` 。

1.4. 如果提取的项目是 `(` 符号，将其推入 `operatorStack` 。

1.5. 如果提取的项目是 `)` 符号，重复处理 `operatorStack` 顶部的操作符，直到遇到 `(` 符号。

阶段 2：清空栈

重复处理 `operatorStack` 顶部的操作符，直到 `operatorStack` 为空。

14.25. Example (示例)

表格详细展示了表达式求值算法的每一步，以及 `operandStack` 和 `operatorStack` 的状态变化。